Task 1: Problem Definition and Scope

Dockerized Bus Seat Reservation System

Student Details	Value
Student Name	DIKSHA PARULEKAR
Roll No.	23102A0068
Branch	CMPN A
Course	DevOps
Project	Dockerized Bus Seat Reservation System
Task	Task 1

1. Objectives

- Define a practical reservation problem and the minimum viable feature boundary.
- Connect application needs to measurable outcomes and testable behavior.
- Identify relevant source files and DevOps tools from the uploaded project.
- Define evidence that should be captured without misrepresenting missing screenshots.
- Separate verified repository facts from items that still require runtime confirmation.

2. Introduction and Project Context

A bus reservation workflow coordinates passengers, bus routes, seat inventory and booking state. The uploaded application is a Java/Spring Boot project named Dockerized Bus Seat Reservation System. Its code includes user, bus, seat and booking entities, REST controllers and service classes. The report scope is an MVP: registration/login, seat availability, booking, confirmation, cancellation and status tracking, with administrator access and a DevOps delivery path.

3. Problem / Requirements / Planning Detail

Pain point	Impact	MVP response
Seat availability is not visible in one place	Passenger may select an unavailable seat	Expose bus and seat availability endpoints
Manual booking records can be inconsistent	Duplicate or unclear reservations	Persist bookings and enforce service-layer checks
Cancellation requires follow-up	Seat may remain unavailable after cancellation	Cancel an active booking and release the seat
No reliable delivery process	Changes can break the app after release	Use version control, tests and repeatable packaging

3.1 Stakeholders

Stakeholder	Responsibility / need
Passenger	Register, sign in, browse seats, book, track and cancel a booking
Administrator	Inspect and manage users, buses, seats and bookings
Developer	Implement APIs, entities, business rules and user interface
QA / tester	Verify registration, booking, cancellation and failure cases
DevOps engineer	Automate build, test, image creation and deployment
Evaluator	Review evidence, report traceability and viva understanding

3.2 Constraints and success criteria

Constraint / criterion	Target or verification method
Scope	Core reservation journeys only; advanced payment and live GPS are not evidenced as MVP features
Duplicate seat booking	Attempt two bookings for one seat; second booking should be

	rejected
Cancellation	Cancel an active booking; status becomes CANCELLED and seat becomes AVAILABLE
Build	Run Maven verification; capture exit status and test summary
UI automation	Run Selenium suite against the configured base URL; preserve actual report output
Deployment	Only mark successful after checking running container/service and health endpoint

4. Architecture and Workflow

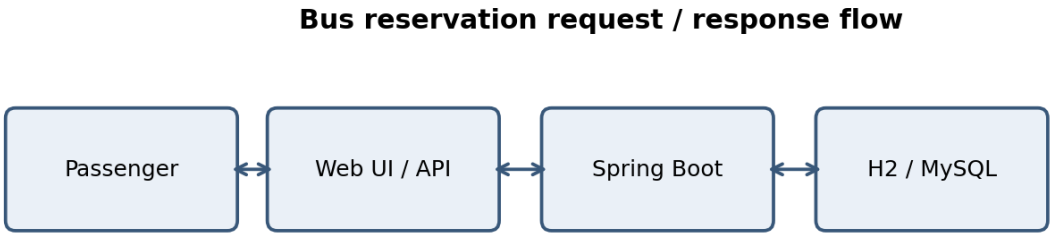


Figure 1.1: Problem-to-system context for the Bus Reservation System

The passenger-facing reservation flow reaches the Spring Boot API and persistence layer. This diagram is logical; exact active ports and database mode must be confirmed from configuration and a running instance.

5. Tools and Technologies

Tool / technology	Verified version / state	Purpose
Java	21 (pom.xml property)	Application language/runtime target
Spring Boot	3.2.4 (Maven parent)	Application framework and REST API
Maven	Version not verified from the ZIP alone	Dependency management, compilation and tests
Spring Data JPA	Dependency present	Persistence abstraction
Spring Security	Dependency present	Authentication and endpoint authorization
H2	Runtime dependency present	Development/test database option
MySQL Connector/J	Runtime dependency present	MySQL connectivity option
Git	Repository metadata included	Version history and branch workflow
Jenkins / Docker / Selenium / Ansible	Files are present; runtime versions not verified	CI, containerization, UI automation and configuration management

6. Implementation Procedure

1. Review the business problem and identify passenger/admin needs.
2. Define in-scope reservation journeys and explicitly list exclusions.
3. Map each journey to an observable acceptance criterion.
4. Use the task backlog and repository files to align scope with implementation.
5. Review measurable checks before implementation starts.

7. Results and Observations

Check	Expected outcome	Actual observation from supplied material	Status

MVP scope documented	Problem, users, boundaries and criteria listed	Scope is described in repository task notes; approval evidence not independently verified	Partially verified
Core reservation entities	User, Bus, Seat and Booking model present	Entity source files are present in ZIP	Verified from source
Runtime behavior	Application starts and main flows work	No live execution performed during report generation	Not verified

Evidence policy: a source file or task tracker entry is not equivalent to a fresh runtime test. Replace 'Not verified' only after capturing the actual command output, application behavior or hosted-tool state.

8. Troubleshooting

Problem	Likely cause	Diagnostic step	Resolution
Scope expands during implementation	New features added without prioritization	Compare request against the frozen MVP	Record as future enhancement or approve a scope change
Seat appears available after cancellation	Booking and seat state are not updated together	Inspect BookingService cancellation path and database state	Update both states within a transaction and add a regression test

9. Advantages, Limitations and Learning

A clear scope reduces uncontrolled feature growth and gives testing a measurable target. The MVP still depends on correct seat-state handling and reliable persistence; payment processing, notifications, production-grade identity and real-time transport integration are outside the documented scope.

10. Conclusion

The report defines the reservation problem, stakeholder needs, constraints and MVP success checks. Source files confirm that the uploaded project contains the core reservation domain and API components. Runtime performance, acceptance by stakeholders and successful end-to-end operation remain to be confirmed with genuine evidence.

11. References

- Spring Boot Reference Documentation: https://docs.spring.io/spring-boot/docs/3.2.4/reference/html/
- Apache Maven Documentation: https://maven.apache.org/guides/
- Spring Security Reference: https://docs.spring.io/spring-security/reference/
- Git Documentation: https://git-scm.com/docs
- GitHub Pull Requests documentation: https://docs.github.com/en/pull-requests
- Selenium Documentation: https://www.selenium.dev/documentation/
- Docker Documentation: https://docs.docker.com/
- Ansible Documentation: https://docs.ansible.com/

Task 2: Agile Planning and DevOps Workflow

Dockerized Bus Seat Reservation System

Student Details	Value
Student Name	DIKSHA PARULEKAR
Roll No.	23102A0068
Branch	CMPN A
Course	DevOps
Project	Dockerized Bus Seat Reservation System
Task	Task 2

1. Objective

Translate the approved scope into actionable work items and verifiable acceptance criteria.

- Connect application needs to measurable outcomes and testable behavior.
- Identify relevant source files and DevOps tools from the uploaded project.
- Define evidence that should be captured without misrepresenting missing screenshots.
- Separate verified repository facts from items that still require runtime confirmation.

2. Introduction and Project Context

Agile planning turns the reservation scope into small units of work with clear acceptance conditions. DevOps connects that planning to source control, automated build, automated testing, packaging, deployment and operational feedback. The uploaded repository includes task notes, a Jenkinsfile, Dockerfile, Selenium test project and Ansible files; the exact success of each tool stage must be established from execution evidence rather than documentation labels alone.

3. Problem / Requirements / Planning Detail

Story ID	User story	Acceptance criteria
US-01	As a passenger, I want to register.	Valid details create one account; duplicate email is rejected with a clear response.
US-02	As a passenger, I want to log in.	Valid credentials authenticate; invalid credentials do not grant access.
US-03	As a passenger, I want to inspect buses and seats.	Bus list and seat statuses load; booked seats cannot be selected.
US-04	As a passenger, I want to reserve a seat.	A booking is created for an available seat; duplicate reservation is prevented.
US-05	As a passenger, I want to track bookings.	The current user's booking list shows status and relevant details.
US-06	As a passenger, I want to cancel.	Only an eligible active booking is cancelled and the seat is released.
US-07	As an administrator, I want oversight.	Administrative endpoints expose system-wide records only to authorized users.

3.1 Sprint / Kanban plan

Sprint	Tasks	Goal	Exit evidence
Sprint 1	1–6	Scope, requirements, repository and MVP workflows	Approved backlog, code history and feature evidence
Sprint 2	7–10	CI pipeline and automated tests	Build logs, test report and quality-gate failure/recovery

Sprint 3	11–15	Container, deployment, configuration and final release	Image/container evidence, Ansible run, health check and release report

3.2 Definition of Done

- Acceptance criteria are demonstrably met.
- Code is committed with a meaningful message and reviewed through the agreed branch process.
- Relevant automated tests pass, with report retained.
- Configuration and deployment instructions are repeatable.
- Evidence is captured and the task status reflects what was actually verified.

4. Architecture and Workflow

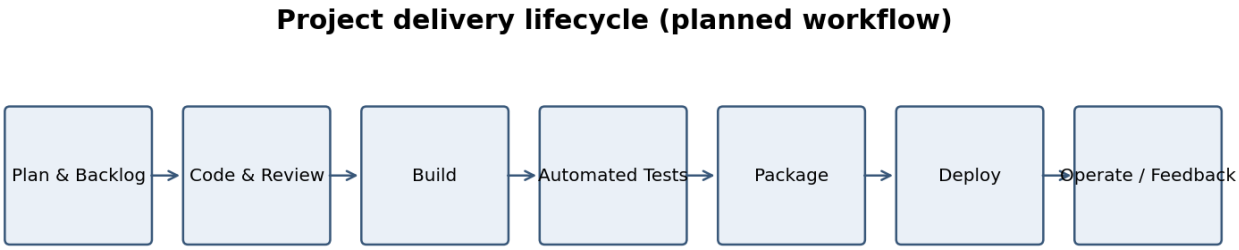


Figure 2.1: Planned Agile and DevOps lifecycle

The lifecycle connects planning, reviewed source changes, build, automated testing, packaging and deployment. A stage is considered successful only when its corresponding tool output is retained.

5. Tools and Technologies

Tool / technology	Verified version / state	Purpose
Java	21 (pom.xml property)	Application language/runtime target
Spring Boot	3.2.4 (Maven parent)	Application framework and REST API
Maven	Version not verified from the ZIP alone	Dependency management, compilation and tests
Spring Data JPA	Dependency present	Persistence abstraction
Spring Security	Dependency present	Authentication and endpoint authorization
H2	Runtime dependency present	Development/test database option
MySQL Connector/J	Runtime dependency present	MySQL connectivity option
Git	Repository metadata included	Version history and branch workflow
Jenkins / Docker / Selenium / Ansible	Files are present; runtime versions not verified	CI, containerization, UI automation and configuration management

6. Implementation Procedure

1. Convert the MVP journeys into user stories with unique IDs.
2. Write observable acceptance criteria for normal and negative paths.
3. Prioritize by core user value and technical dependency.
4. Group the 15 assigned tasks into three delivery sprints.
5. Move a card to Done only when code, tests and evidence meet the Definition of Done.

7. Results and Observations

Check	Expected outcome	Actual observation from supplied material	Status
User stories	Stories have acceptance criteria	Task planning markdown exists; stories inspected	Verified from documentation
15-task delivery plan	All official tasks mapped	Official task list contains 15 tasks	Verified from supplied task list
Agile board activity	Real board reflects task status	No live GitHub board access evidence supplied	Not verified

Evidence policy: a source file or task tracker entry is not equivalent to a fresh runtime test. Replace 'Not verified' only after capturing the actual command output, application behavior or hosted-tool state.

8. Troubleshooting

Problem	Likely cause	Diagnostic step	Resolution
Stories are too broad	Acceptance criteria are not measurable	Check whether each criterion can be tested	Split story and use observable outcomes
Cards remain Done despite failing tests	Definition of Done is weakly enforced	Review test reports and merge checks	Require test/evidence before closing task

9. Advantages, Limitations and Learning

Agile planning makes work visible and encourages small feedback loops. The task tracker in the ZIP states all tasks are complete, but that status should be backed by actual build, test and deployment evidence before submission.

10. Conclusion

The project is organized as a 15-task DevOps effort with user stories, acceptance conditions and a lightweight sprint structure. The next step is to enforce the Definition of Done with actual automated results and task-specific evidence rather than relying only on status notes.

11. References

- Spring Boot Reference Documentation: https://docs.spring.io/spring-boot/docs/3.2.4/reference/html/
- Apache Maven Documentation: https://maven.apache.org/guides/
- Spring Security Reference: https://docs.spring.io/spring-security/reference/
- Git Documentation: https://git-scm.com/docs
- GitHub Pull Requests documentation: https://docs.github.com/en/pull-requests
- Selenium Documentation: https://www.selenium.dev/documentation/
- Docker Documentation: https://docs.docker.com/
- Ansible Documentation: https://docs.ansible.com/

Task 3: Requirements, Architecture and Technology Setup

Dockerized Bus Seat Reservation System

Student Details	Value
Student Name	DIKSHA PARULEKAR
Roll No.	23102A0068
Branch	CMPN A
Course	DevOps
Project	Dockerized Bus Seat Reservation System
Task	Task 3

1. Objective

- Specify the system requirements, architecture boundaries, data model and local setup.
- Connect application needs to measurable outcomes and testable behavior.
- Identify relevant source files and DevOps tools from the uploaded project.
- Define evidence that should be captured without misrepresenting missing screenshots.
- Separate verified repository facts from items that still require runtime confirmation.

2. Introduction and Project Context

The project uses a layered Spring Boot application with REST controllers, service logic, JPA entities/repositories and Spring Security. The Maven descriptor verifies Spring Boot 3.2.4 and Java 21. H2 and MySQL runtime dependencies are present; the active runtime database depends on application configuration and environment variables, so this report does not assume production MySQL is currently configured.

3. Problem / Requirements / Planning Detail

Requirement ID	Requirement	Verification
FR-01	Register a user and reject duplicate email.	Service test for valid and duplicate registration.
FR-02	Authenticate user credentials securely.	Authentication tests and protected endpoint check.
FR-03	List buses and seats with availability status.	GET bus and seat endpoints return data.
FR-04	Book an available seat and prevent duplicate booking.	Booking service test and repeated booking attempt.
FR-05	List bookings for the authenticated user.	GET /api/bookings/my returns only caller's records.
FR-06	Cancel active booking and release seat.	PUT /api/bookings/{id}/cancel; verify booking and seat state.
FR-07	Provide administrative booking visibility.	Admin endpoint access control test.

3.1 Architecture and data model

Entity	Purpose	Key relationship
User	Identity, email, password hash and role	One user may own multiple bookings
Bus	Route and departure information	One bus has multiple seats
Seat	Seat number and availability	Belongs to a bus; may be referenced by booking
Booking	Reservation record, date and status	References the user and seat

Figure 3.1 shows the logical request path. This is an architecture view derived from the checked-in Spring Boot package layout; deployment-specific addresses and active database settings must be verified in the runtime configuration.

4. Architecture and Workflow

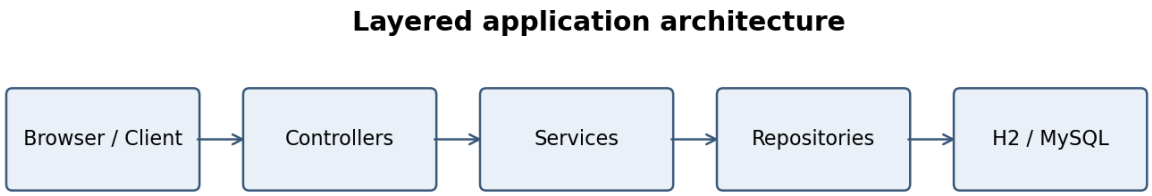


Figure 3.1: Logical layered architecture of the application

The controller/service/repository separation makes HTTP handling distinct from business rules and persistence. The actual Java package contains controller, service, repository and entity classes consistent with this logical model.

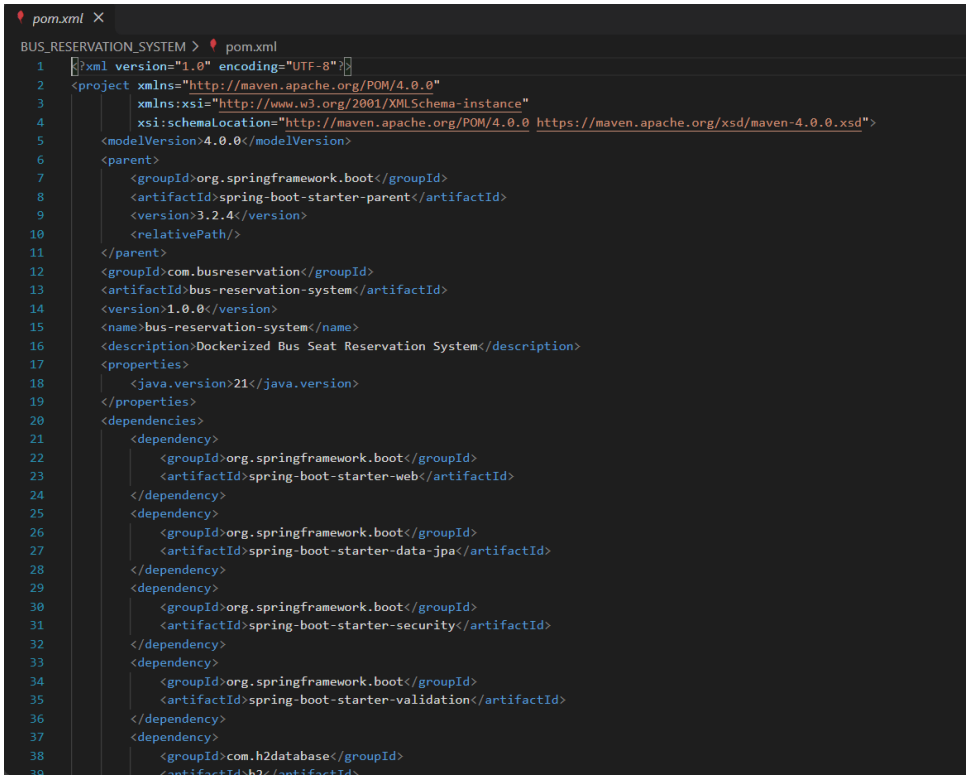
5. Tools and Technologies

Tool / technology	Verified version / state	Purpose
Java	21 (pom.xml property)	Application language/runtime target
Spring Boot	3.2.4 (Maven parent)	Application framework and REST API
Maven	Version not verified from the ZIP alone	Dependency management, compilation and tests
Spring Data JPA	Dependency present	Persistence abstraction
Spring Security	Dependency present	Authentication and endpoint authorization
H2	Runtime dependency present	Development/test database option
MySQL Connector/J	Runtime dependency present	MySQL connectivity option
Git	Repository metadata included	Version history and branch workflow
Jenkins / Docker / Selenium / Ansible	Files are present; runtime versions not verified	CI, containerization, UI automation and configuration management

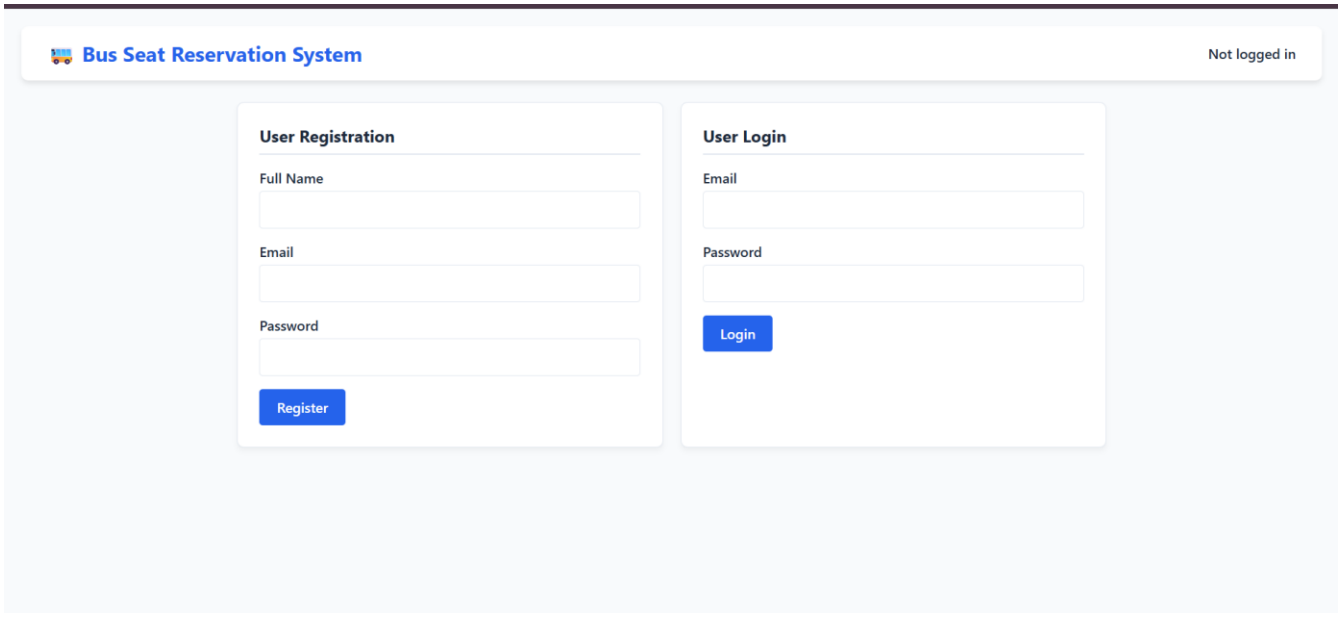
6. Implementation Procedure

1. Inspect pom.xml to establish the framework and Java target.
2. Map controller, service, repository and entity packages.
3. Define functional and non-functional requirements.
4. Document the User, Bus, Seat and Booking relationships.
5. Run the application and verify routes/database behavior only in the target environment.

7. Evidence and Screenshot Placement



Maven and Java configuration
Explanation: Verifies technology versions from source.



Local application verification
Explanation: Confirms the environment actually starts.

8. Results and Observations

Check	Expected outcome	Actual observation from supplied material	Status
Framework version	Spring Boot 3.2.4	pom.xml parent declares 3.2.4	Verified from source
Java target	Java 21	pom.xml declares java.version 21	Verified from source
Build/test result	Maven verification succeeds	No fresh build executed in this report batch	Not verified

Evidence policy: a source file or task tracker entry is not equivalent to a fresh runtime test. Replace 'Not verified' only after capturing the actual command output, application behavior or hosted-tool state.

9. Troubleshooting

Problem	Likely cause	Diagnostic step	Resolution
Maven uses wrong Java	JAVA_HOME or installed JDK differs	Run java -version and mvn -version	Select JDK 21 and re-run Maven
Database connection fails	Active profile or datasource settings mismatch	Inspect application properties and startup logs	Set the intended profile and verify dependency/configuration

10. Advantages, Limitations and Learning

A layered Spring Boot design separates HTTP concerns, business rules and persistence, which makes unit testing and maintenance easier. H2 and MySQL dependencies are present, but the active datasource, deployment environment and exact API security behavior should be verified at runtime.

11. Conclusion

The checked-in Maven descriptor establishes Java 21 and Spring Boot 3.2.4, and the source tree provides the expected controller/service/entity structure. A fresh local build, runtime configuration review and endpoint verification are still needed to confirm the complete environment.

12. References

- Spring Boot Reference Documentation: https://docs.spring.io/spring-boot/docs/3.2.4/reference/html/
- Apache Maven Documentation: https://maven.apache.org/guides/
- Spring Security Reference: https://docs.spring.io/spring-security/reference/
- Git Documentation: https://git-scm.com/docs
- GitHub Pull Requests documentation: https://docs.github.com/en/pull-requests
- Selenium Documentation: https://www.selenium.dev/documentation/
- Docker Documentation: https://docs.docker.com/
- Ansible Documentation: https://docs.ansible.com/

Task 4: Git and GitHub Repository Initialization

Dockerized Bus Seat Reservation System

Student Details	Value
Student Name	DIKSHA PARULEKAR
Roll No.	23102A0068
Branch	CMPN A
Course	DevOps
Project	Dockerized Bus Seat Reservation System
Task	Task 4

1. Objective

- Establish a traceable Git/GitHub repository and collaboration conventions.
- Connect application needs to measurable outcomes and testable behavior.
- Identify relevant source files and DevOps tools from the uploaded project.
- Define evidence that should be captured without misrepresenting missing screenshots.
- Separate verified repository facts from items that still require runtime confirmation.

2. Introduction and Project Context

Git records source history, while GitHub hosts the shared repository and enables pull requests and review. A disciplined branch policy reduces accidental changes to the release branch and gives each feature a traceable commit history. The uploaded ZIP contains a nested Git repository and branch/log metadata, but local repository metadata alone does not prove a remote pull request was actually reviewed or merged on GitHub.

3. Problem / Requirements / Planning Detail

Repository item	Observed role	Verification note
README.md	Project description and usage notes	Read before first clone/build
.gitignore	Excludes generated/IDE files where configured	Inspect tracked files before commit
pom.xml	Build and dependency definition	Java 21 property and Spring Boot 3.2.4 verified
Jenkinsfile	Pipeline-as-code entry point	Review actual stages before claiming CI behavior
Dockerfile	Container image instructions	Build locally and inspect image metadata
ansible/site.yml	Configuration automation playbook	Check inventory and variables before running
docs/	Task notes and viva documentation	Supporting documentation, not runtime proof

3.1 Branching policy

Branch	Purpose	Rule
main	Release-ready baseline	Merge reviewed changes only; tag releases
develop	Integration branch	Receive tested feature pull requests
feature/<short-name>	Isolated work	Branch from develop; commit focused changes; open PR

Recommended commit prefixes: feat (feature), fix (bug), docs (documentation), test (tests), ci (pipeline) and chore (maintenance).

4. Architecture and Workflow

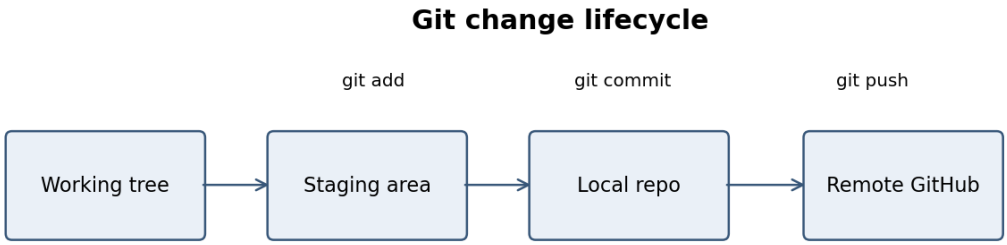


Figure 4.1: Git working tree to remote repository flow

The normal Git path is working tree → staging → local commit → remote push. GitHub pull requests add review and merge controls around branch integration.

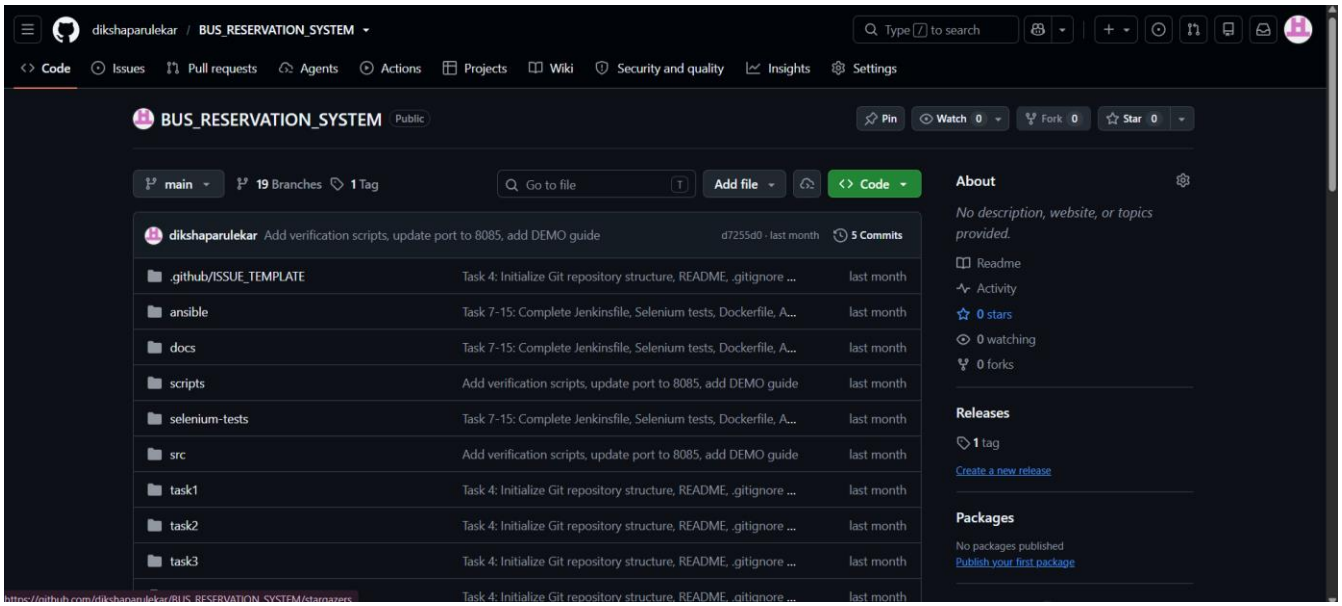
5. Tools and Technologies

Tool / technology	Verified version / state	Purpose
Java	21 (pom.xml property)	Application language/runtime target
Spring Boot	3.2.4 (Maven parent)	Application framework and REST API
Maven	Version not verified from the ZIP alone	Dependency management, compilation and tests
Spring Data JPA	Dependency present	Persistence abstraction
Spring Security	Dependency present	Authentication and endpoint authorization
H2	Runtime dependency present	Development/test database option
MySQL Connector/J	Runtime dependency present	MySQL connectivity option
Git	Repository metadata included	Version history and branch workflow
Jenkins / Docker / Selenium / Ansible	Files are present; runtime versions not verified	CI, containerization, UI automation and configuration management

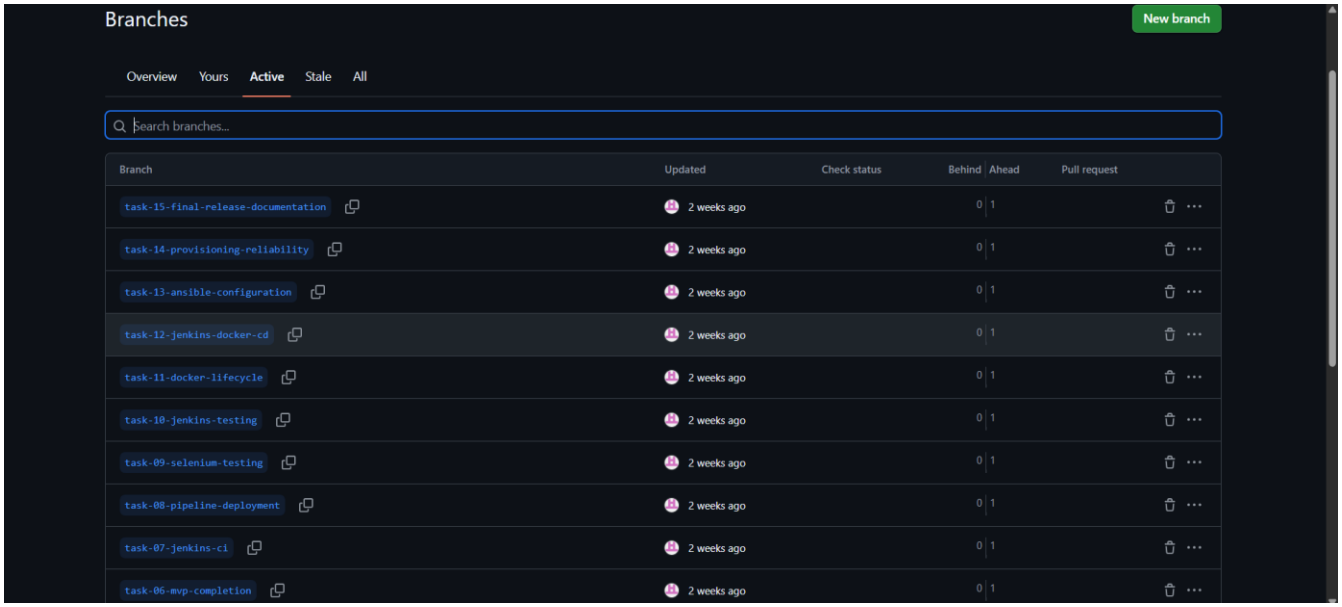
6. Implementation Procedure

1. Inspect the current Git state using git status and git log.
2. Confirm .gitignore excludes build output and secrets.
3. Check README setup instructions and repository remote.
4. Use main, develop and feature branches consistently.
5. Open a pull request and preserve review/merge evidence.

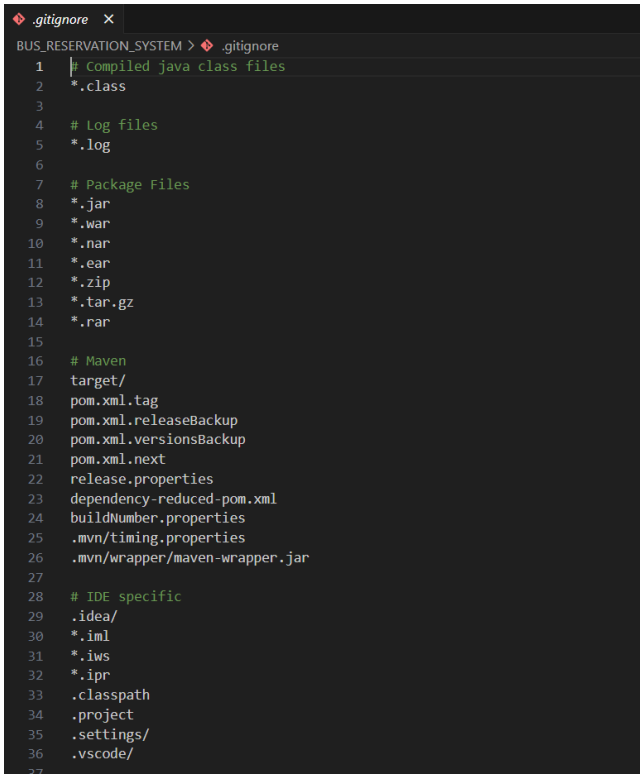
7. Evidence and Screenshot Placement



Local repository history
Explanation: Provides local source-control evidence.



Remote repository structure
Explanation: Proves remote hosting only when captured from the real account.



```
.gitignore
BUS_RESERVATION_SYSTEM > .gitignore
# Compiled java class files
*.class

# Log files
*.log

# Package Files
*.jar
*.war
*.nar
*.ear
*.zip
*.tar.gz
*.rar

# Maven
target/
pom.xml.tag
pom.xml.releaseBackup
pom.xml.versionsBackup
pom.xml.next
release.properties
dependency-reduced-pom.xml
buildNumber.properties
.mvn/timing.properties
.mvn/wrapper/maven-wrapper.jar

# IDE specific
.idea/
*.iml
*.iws
*.ipr
.classpath
.project
.settings/
.vscode/
```

8. Results and Observations

Check	Expected outcome	Actual observation from supplied material	Status
Repository files	README, build and DevOps files present	Relevant files present in extracted project ZIP	Verified from source archive
Branch metadata	Feature/task branches exist	ZIP contains nested Git metadata and branch names	Partially verified
Remote PR/review	A real reviewed PR exists	Cannot prove from source ZIP alone	Not verified

Evidence policy: a source file or task tracker entry is not equivalent to a fresh runtime test. Replace 'Not verified' only after capturing the actual command output, application behavior or hosted-tool state.

9. Troubleshooting

Problem	Likely cause	Diagnostic step	Resolution
Push rejected	Remote branch has new commits or authentication issue	Run git status and git remote -v; read push error	Pull/rebase safely and use supported GitHub authentication
Build output committed	Ignore rules missing or file already tracked	Run git status --short and git ls-files target	Correct .gitignore and untrack generated artifacts

10. Advantages, Limitations and Learning

Git provides history and supports safe parallel development. Repository metadata can support local history claims but cannot alone prove GitHub-side review, branch protection or remote release behavior.

11. Conclusion

The repository archive contains project source, documentation and DevOps configuration files. GitHub collaboration should be evidenced with the actual remote URL, branch list, PR review and merge history; these hosted events are not asserted solely from local files.

12. References

- Spring Boot Reference Documentation: https://docs.spring.io/spring-boot/docs/3.2.4/reference/html/
- Apache Maven Documentation: https://maven.apache.org/guides/
- Spring Security Reference: https://docs.spring.io/spring-security/reference/
- Git Documentation: https://git-scm.com/docs
- GitHub Pull Requests documentation: https://docs.github.com/en/pull-requests
- Selenium Documentation: https://www.selenium.dev/documentation/
- Docker Documentation: https://docs.docker.com/
- Ansible Documentation: https://docs.ansible.com/

Task 5: Feature Development with Branching

Dockerized Bus Seat Reservation System

Student Details	Value
Student Name	DIKSHA PARULEKAR
Roll No.	23102A0068
Branch	CMPN A
Course	DevOps
Project	Dockerized Bus Seat Reservation System
Task	Task 5

1. Objective

- Demonstrate an isolated feature change and a reviewed path back to the integration branch.
- Connect application needs to measurable outcomes and testable behavior.
- Identify relevant source files and DevOps tools from the uploaded project.
- Define evidence that should be captured without misrepresenting missing screenshots.
- Separate verified repository facts from items that still require runtime confirmation.

2. Introduction and Project Context

Feature branching keeps an implementation change isolated until it is reviewed and integrated. The uploaded project contains multiple feature/task branch names in Git metadata and a source-level feature implementation. For an academically defensible record, commands and screenshot placeholders are provided; a pull request, review and merge are described as evidence requirements unless independently visible in the supplied files.

3. Problem / Requirements / Planning Detail

Step	Action	Why it matters
1	Synchronize develop and create feature branch.	Starts from the shared integration baseline.
2	Implement one small reservation feature.	Keeps the review scope narrow.
3	Run targeted tests and inspect git diff.	Catches mistakes before publication.
4	Commit with a meaningful message.	Creates a traceable change unit.
5	Push branch and open a PR to develop.	Makes review and collaboration visible.
6	Resolve feedback, rerun tests, merge and pull.	Ensures integration includes reviewed code.

3.1 Git command sequence

```
git status
git switch develop
git pull --ff-only
git switch -c feature/booking-workflow
# edit files, then review the change
git diff
git add src/
git commit -m "feat: implement booking workflow"
git push -u origin feature/booking-workflow
```

The example commit message is a recommended template, not a claim that this exact commit was executed. Use the repository's actual branch name and commit ID in the final evidence.

4. Architecture and Workflow

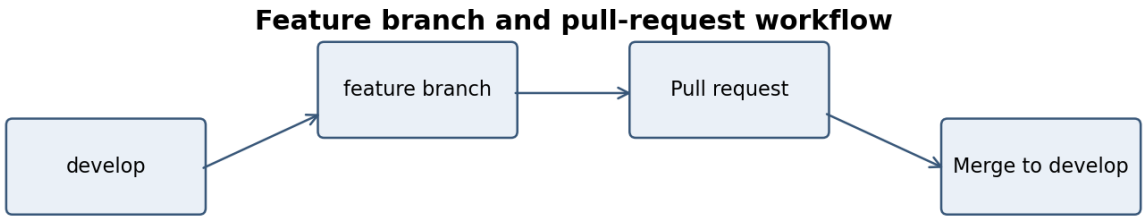


Figure 5.1: Feature branch and pull-request integration flow

A feature branch is published for review before merge. The diagram shows the intended process; a screenshot of the actual PR and merge is required to prove that it occurred.

5. Tools and Technologies

Tool / technology	Verified version / state	Purpose
Java	21 (pom.xml property)	Application language/runtime target
Spring Boot	3.2.4 (Maven parent)	Application framework and REST API
Maven	Version not verified from the ZIP alone	Dependency management, compilation and tests
Spring Data JPA	Dependency present	Persistence abstraction
Spring Security	Dependency present	Authentication and endpoint authorization
H2	Runtime dependency present	Development/test database option
MySQL Connector/J	Runtime dependency present	MySQL connectivity option
Git	Repository metadata included	Version history and branch workflow
Jenkins / Docker / Selenium / Ansible	Files are present; runtime versions not verified	CI, containerization, UI automation and configuration management

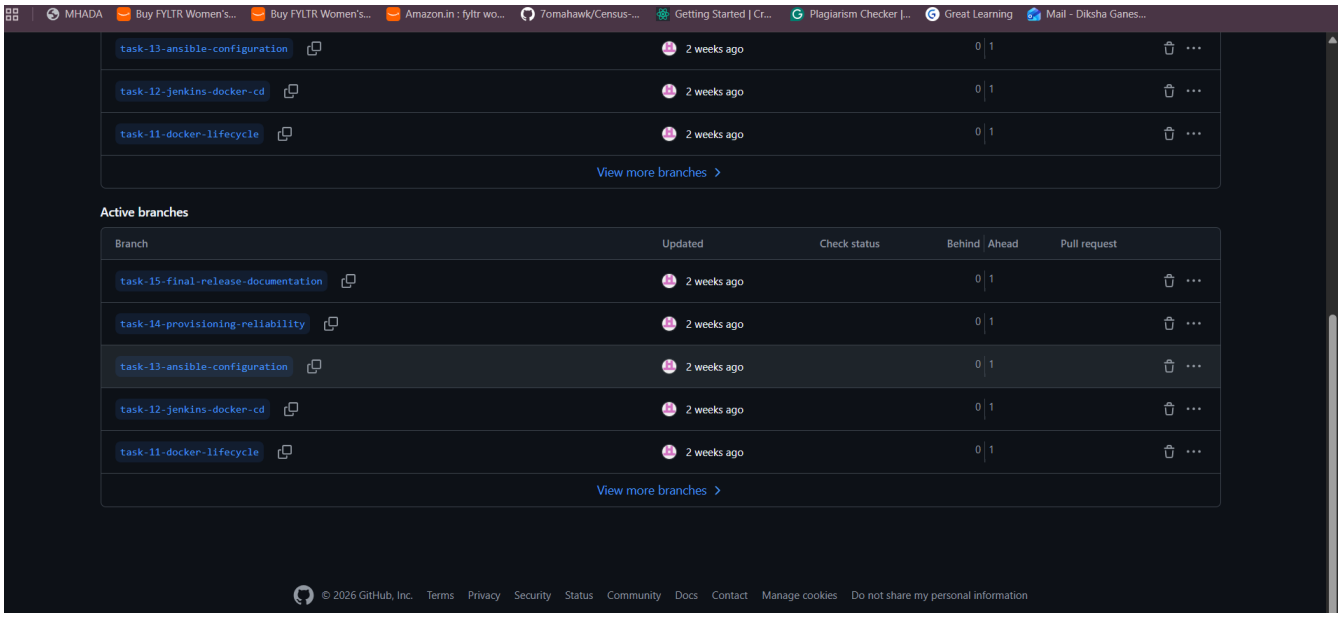
6. Implementation Procedure

1. Start from an up-to-date develop branch.
2. Create a feature branch named for the reservation change.
3. Implement one focused change and inspect git diff.
4. Run the relevant Maven tests before commit.
5. Push the branch, request review, address comments, merge and record the actual commit/PR identifiers.

7. Evidence and Screenshot Placement

```
Problems  Output  Debug Console  Terminal  Ports

PS D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM> git branch --show-current
main
```



8. Results and Observations

Check	Expected outcome	Actual observation from supplied material	Status
Feature branch change	Focused change exists on a branch	Branch metadata is present; specific PR not independently verified	Partially verified
Review and merge	PR review and merge commit exist	Requires live GitHub screenshot/remote history	Not verified
Automated checks	Relevant tests pass	Tests exist; no fresh run performed in report generation	Not verified

Evidence policy: a source file or task tracker entry is not equivalent to a fresh runtime test. Replace 'Not verified' only after capturing the actual command output, application behavior or hosted-tool state.

9. Troubleshooting

Problem	Likely cause	Diagnostic step	Resolution
Merge conflict	Two branches edit the same lines	Run git status and inspect conflict markers	Resolve deliberately, test, then commit the resolution
PR cannot be merged	Checks or reviews incomplete	Inspect actual PR status and required checks	Address feedback and rerun checks before merging

10. Advantages, Limitations and Learning

Feature branches isolate changes and create a review checkpoint. The quality of the workflow depends on meaningful commits, actual review, automated tests and documented merge results; screenshots must come from the real repository.

11. Conclusion

The feature-branch workflow provides a repeatable pattern for isolated implementation and review. The report deliberately leaves PR and test outcomes unverified where the supplied files do not independently prove them.

12. References

- Spring Boot Reference Documentation: https://docs.spring.io/spring-boot/docs/3.2.4/reference/html/
- Apache Maven Documentation: https://maven.apache.org/guides/

- Spring Security Reference: https://docs.spring.io/spring-security/reference/
- Git Documentation: https://git-scm.com/docs
- GitHub Pull Requests documentation: https://docs.github.com/en/pull-requests
- Selenium Documentation: https://www.selenium.dev/documentation/
- Docker Documentation: https://docs.docker.com/
- Ansible Documentation: https://docs.ansible.com/

Task 06: MVP Completion and Git Collaboration

DevOps Bus Reservation System

Student Name	DIKSHA PARULEKAR
Roll No.	23102A0068
Branch	CMPN A
Course	DevOps
Project	DevOps Bus Reservation System
Task	Task 06

1. Objective

Review the implemented Bus Reservation System MVP, map its user-facing workflows to the source code, and define a safe Git collaboration workflow for completing and integrating changes. Repository contents are used as evidence of implementation; commands below are instructions unless an existing Git record explicitly verifies them.

2. MVP Scope and Implemented Features

The README describes a web-based Bus Seat Reservation System with registration and login, bus and seat availability, seat selection, booking, booking history/status, and cancellation. The Java backend uses Spring Boot, Spring Data JPA and Spring Security; the browser interface uses HTML, CSS and vanilla JavaScript with Fetch API requests.

Feature	Source-backed implementation / verification point
Registration and login	UserService, RegisterRequest/LoginRequest DTOs and Spring Security dependencies; verify the registration and login endpoints and BCrypt handling in the source before a live demonstration.
Bus listing and seat availability	BusService and Seat entity/service flow; sample bus data is initialized by busService.initSampleData() in the application test setup.
Seat selection and booking	BookingService.createBooking(...) creates a reservation and changes the selected seat to BOOKED; the unit/integration test checks CONFIRMED status.
Duplicate booking prevention	BookingService test expects a RuntimeException containing “already booked” on a second booking attempt for the same seat.
Booking cancellation	BookingService.cancelBooking(...) is tested to set booking status to CANCELLED and return the seat to AVAILABLE.
Booking history/status	Documented as an MVP capability in README; verify the current UI/API flow during execution before claiming live behaviour.

The project is configured for H2 in-memory development/testing and has MySQL connector support for deployment configuration. In-memory H2 data should be treated as temporary and may reset when the application restarts.

3. Relevant Repository Structure

Path	Purpose
pom.xml	Spring Boot 3.2.4 parent, Java 21 property, application dependencies and Maven build.
src/main/java/com/busreservation/	Application classes, entities, DTOs, services and web/security code.
src/main/resources/application.properties	Port 8085 and environment-overridable datasource configuration; H2 default and H2 console path.
src/main/resources/static/	Static browser assets, including vanilla JavaScript and CSS where present.
src/test/java/com/busreservation/Bu	Spring Boot tests for registration/login, duplicate registration, bus/seat availability,

sReservationApplicationTests.java	booking/duplicate booking and cancellation.
selenium-tests/	Separate Maven project and Selenium end-to-end test source.
Jenkinsfile	Declarative pipeline with Checkout, Build, Test and Package; uses Windows bat commands.
Dockerfile, settings.xml, ansible/site.yml	Container build, Maven settings and Ansible deployment/health-check example.

```
Project root: BUS_RESERVATION_SYSTEM/BUS_RESERVATION_SYSTEM
Key build: mvn clean package
Application configuration: src/main/resources/application.properties
Default configured application port: 8085
README quick-start example references http://localhost:8080; reconcile the documented URL with
application.properties before testing.
```

4. MVP Verification Procedure

1. Review README.md and inspect the application source; do not infer behaviour from the README alone.
2. From the project root, run java -version and mvn -version. Confirm Java 21 and a compatible Maven installation.
3. Run mvn clean test, then start the app using the project's documented command or java -jar target/bus-reservation-system-1.0.0.jar after packaging.
4. Use the configured server port from application.properties (currently 8085 unless overridden). Confirm the actual listening port in startup logs.
5. Register a test user, log in, open the bus list, inspect seat availability, attempt one booking, confirm the booking status, and cancel it. Capture actual UI and terminal evidence.
6. Repeat the duplicate-booking case with the same seat and confirm the application rejects the second reservation. Use a disposable test dataset.

The source ZIP includes a Maven Surefire text report for the application test class that records 5 tests with 0 failures and 0 errors, and a Selenium report file that records 7 tests with 0 failures and 0 errors. These are pre-existing artifacts packaged in the ZIP, not proof that tests were freshly run against the current source at report-generation time. Re-run the commands and capture current output before reporting a verified result.

5. Git Collaboration Workflow

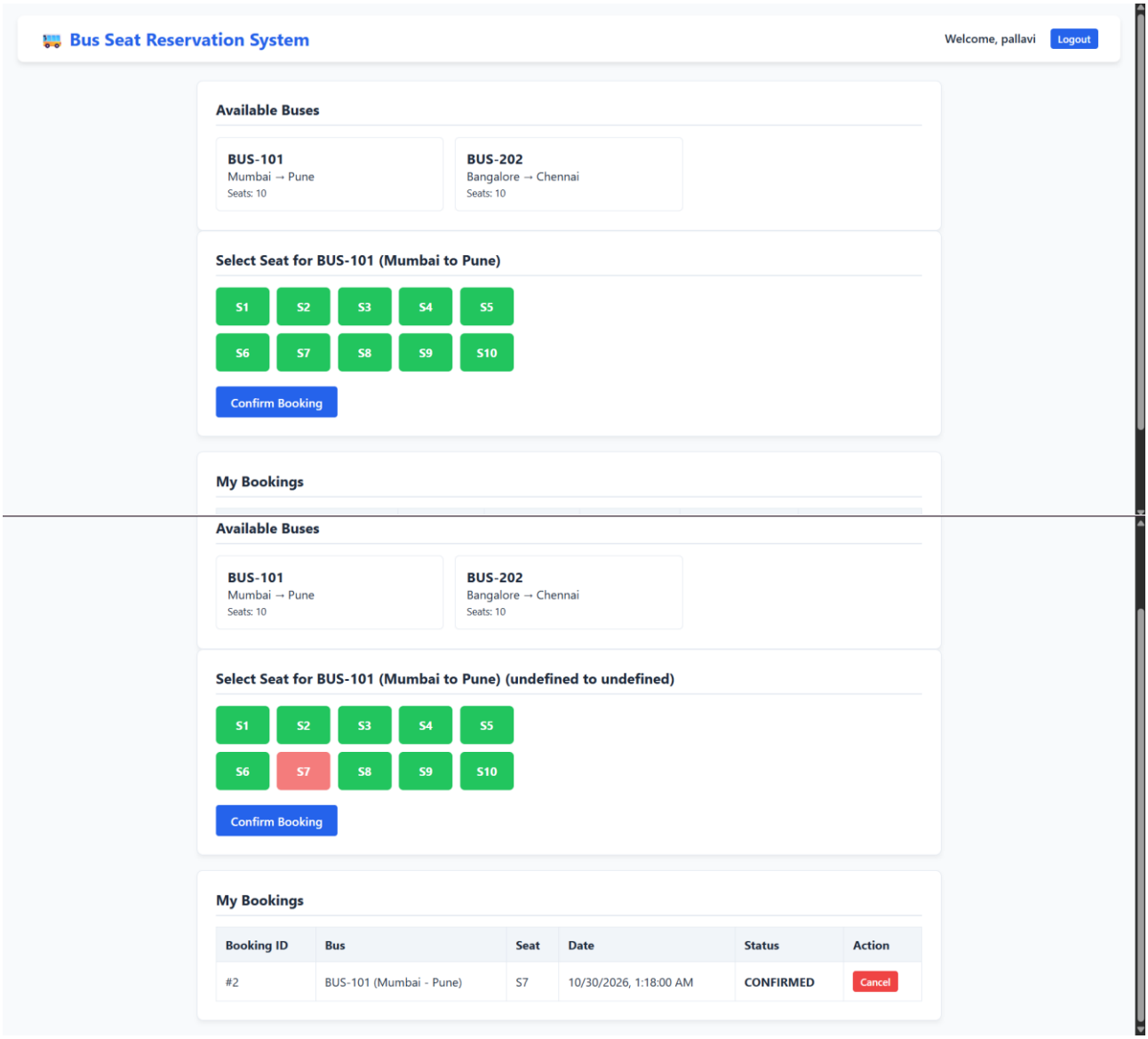
Step	Recommended workflow
1. Branch	Update develop, then create a task branch such as feature/booking-cancellation or fix/seat-race-condition.
2. Change and test	Make a focused change; run relevant Maven tests and record the result.
3. Commit	Use a meaningful message such as feat: add booking cancellation validation or test: cover duplicate seat booking.
4. Push	Push the feature branch to the configured remote; do not include credentials or local secrets.
5. Pull request	Open a PR targeting develop, summarize behaviour, list tests, and attach genuine screenshots/logs.
6. Review and merge	Resolve reviewer comments, obtain review, and merge only after required checks pass.
7. Conflict resolution	Fetch latest develop, merge/rebase as team policy allows, edit conflict markers carefully, run git diff and tests, then commit the resolution.
8. Release	Merge reviewed work to main according to the project policy and create a release tag only after verification.

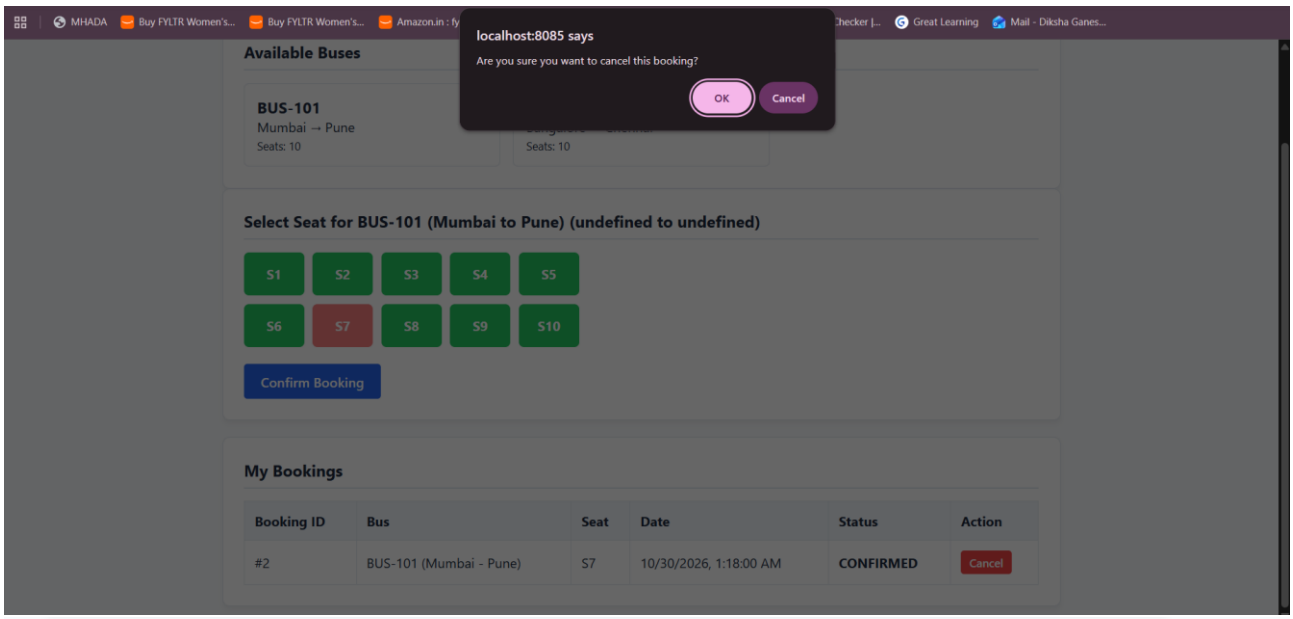
The ZIP contains Git metadata and branch/log directories with task-oriented branch names. This report does not claim that a particular PR was reviewed or merged, that a remote is accessible, or that a release tag exists unless the student captures current git log, git branch -a, git remote -v (redact private URLs if needed), and tag output.

```
git status --short --branch
git branch -a
```

```
git log --oneline --decorate --graph -20
git diff --check
# Example workflow; replace branch name with the actual task
git switch develop
git pull
git switch -c feature/booking-cancellation
# edit files and run tests
git add <changed-files>
git commit -m "feat: improve booking cancellation"
git push -u origin feature/booking-cancellation
```

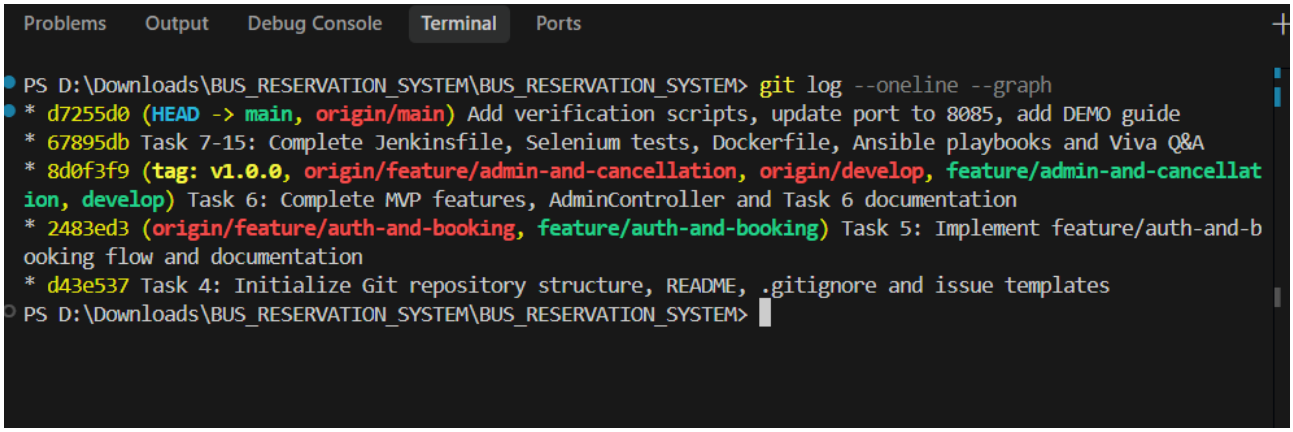
6. Evidence and Screenshots

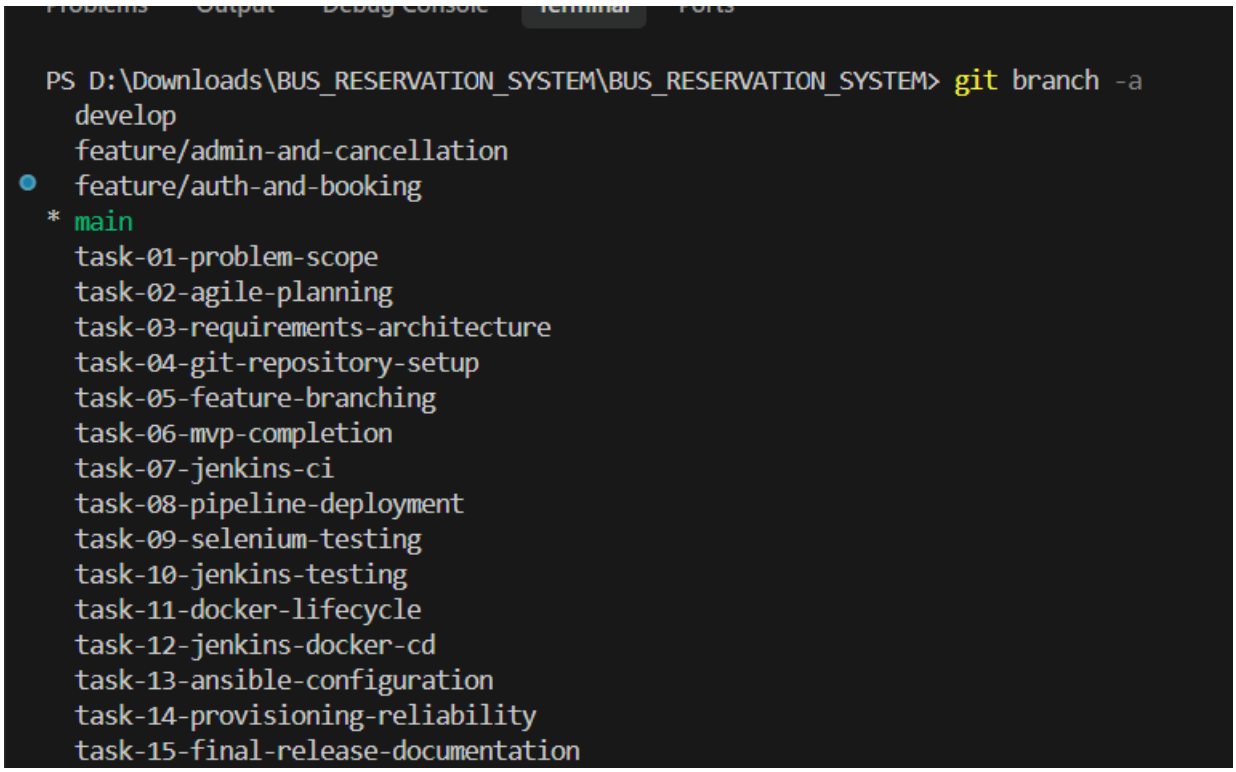




My Bookings

Booking ID	Bus	Seat	Date	Status	Action
#2	BUS-101 (Mumbai - Pune)	S7	10/30/2026, 1:18:00 AM	**CANCELLED**	-





7. Result and Conclusion

The repository contains the core application code, a Maven test class, a separate Selenium project, a Jenkinsfile, a Dockerfile and Ansible configuration. The described MVP covers user authentication, bus/seat availability, booking and cancellation. Current runtime verification and the collaboration actions must be completed and evidenced on the student's machine; source files and bundled reports alone do not prove a fresh successful run.

References: project README.md; pom.xml; application.properties; BusReservationApplicationTests.java; ReservationSystemE2ETest.java; Jenkinsfile.

Task 07: Jenkins Installation and Continuous Integration Job

DevOps Bus Reservation System

Student Name	DIKSHA PARULEKAR
Roll No.	23102A0068
Branch	CMPN A
Course	DevOps
Project	DevOps Bus Reservation System
Task	Task 07

1. Objective

Prepare Jenkins for the Bus Reservation System and configure a continuous integration job that checks out the repository, compiles the application, runs Maven tests, packages the application and archives the generated JAR. Installation and successful execution must be confirmed on the target machine.

2. Prerequisites and Environment

Requirement	Project-specific detail
Java	Project pom.xml declares Java 21; use a JDK 21 installation for the Jenkins agent.
Maven	Maven must be available to the Jenkins agent, either through PATH or a configured Jenkins tool.
Source control	Git installed on the agent and repository credentials configured securely if the repository is private.
Jenkins agent	The current Jenkinsfile uses Windows bat steps, so run it on a Windows agent or change commands for the target OS.
Maven settings	Jenkinsfile hard-codes D:\maven-settings.xml as MAVEN_SETTINGS. Confirm the file exists on the Windows agent or replace it with a managed settings file/path.
Artifact	Jenkinsfile archives target/*.jar and allows an empty archive; confirm the expected JAR is present after packaging.

The project's application is configured on port 8085 in application.properties. CI compilation/tests/package do not themselves prove the application was deployed or started.

3. Jenkins Installation and Configuration

1. Install Jenkins using the official installer/package for the operating system used by the student. Follow the installer's instructions for the required Java runtime for that Jenkins release.
2. Open the local Jenkins URL printed by the installer or service configuration. Unlock Jenkins using the locally generated initial administrator password; never place the password in this report.
3. Install or confirm the recommended plugins, including Pipeline, Git and the appropriate credentials integration. Install a Maven integration plugin only if using Jenkins-managed Maven tools.
4. Under Manage Jenkins → Tools, configure the JDK 21 installation and Maven, or verify java and mvn are available in the agent environment.
5. Under Manage Jenkins → Nodes, ensure the selected agent is online and that it matches the pipeline command type. A Windows agent is required by the current bat steps.
6. Create or verify the Maven settings file path D:\maven-settings.xml. If unavailable, edit the Jenkinsfile to use a valid path or Jenkins-managed settings; do not create a fake settings file merely to satisfy the script.

7. Configure repository credentials through Jenkins Credentials when needed. Do not hard-code passwords or access tokens in the Jenkinsfile.

Installation status is not known from the uploaded source. Treat all steps in this section as a procedure until the student captures the Jenkins dashboard and a real build.

4. Create the CI Job

8. Select New Item, enter a descriptive job name such as Bus-Reservation-CI (example only), choose Pipeline, and continue.
9. Under Pipeline, choose Pipeline script from SCM, select Git, enter the actual repository URL, select credentials if required, and choose the real branch to build.
10. Set Script Path to Jenkinsfile at the repository root.
11. Save the job and select Build Now. Open Console Output and inspect the Checkout, Build, Test and Package stages.
12. After a run, open the build page and confirm the archived artifact link. If the job fails, use the first meaningful error in Console Output rather than treating a red status as a Jenkins installation problem.
13. Configure a webhook or polling trigger only after repository access is confirmed. The Jenkinsfile itself does not define a webhook, pollSCM trigger, or automatic trigger policy.

Pipeline stage	Actual command/configuration	Purpose
Checkout	checkout scm	Checks out the source associated with the Pipeline job.
Build	bat "mvn -s ${env.MAVEN_SETTINGS} clean compile"	Compiles application source on a Windows agent.
Test	bat "mvn -s ${env.MAVEN_SETTINGS} test"	Runs the root Maven test lifecycle.
Package	bat "mvn -s ${env.MAVEN_SETTINGS} package -DskipTests"	Packages the application while skipping a second test execution.
Post: always	archiveArtifacts artifacts: 'target/*.jar', allowEmptyArchive: true	Attempts to archive JAR files even when earlier stages fail; allowEmptyArchive means absence may not fail the build.

5. Build and Test Verification

```
java -version
mvn -version
# From the repository root, outside Jenkins
mvn -s D:\maven-settings.xml clean compile
mvn -s D:\maven-settings.xml test
mvn -s D:\maven-settings.xml package -DskipTests
```

Run the commands locally only after confirming the settings path exists. For a first diagnostic run, Maven can also be invoked without -s if the project does not require custom repository settings. The exact Jenkinsfile currently invokes the custom settings path in every Maven command.

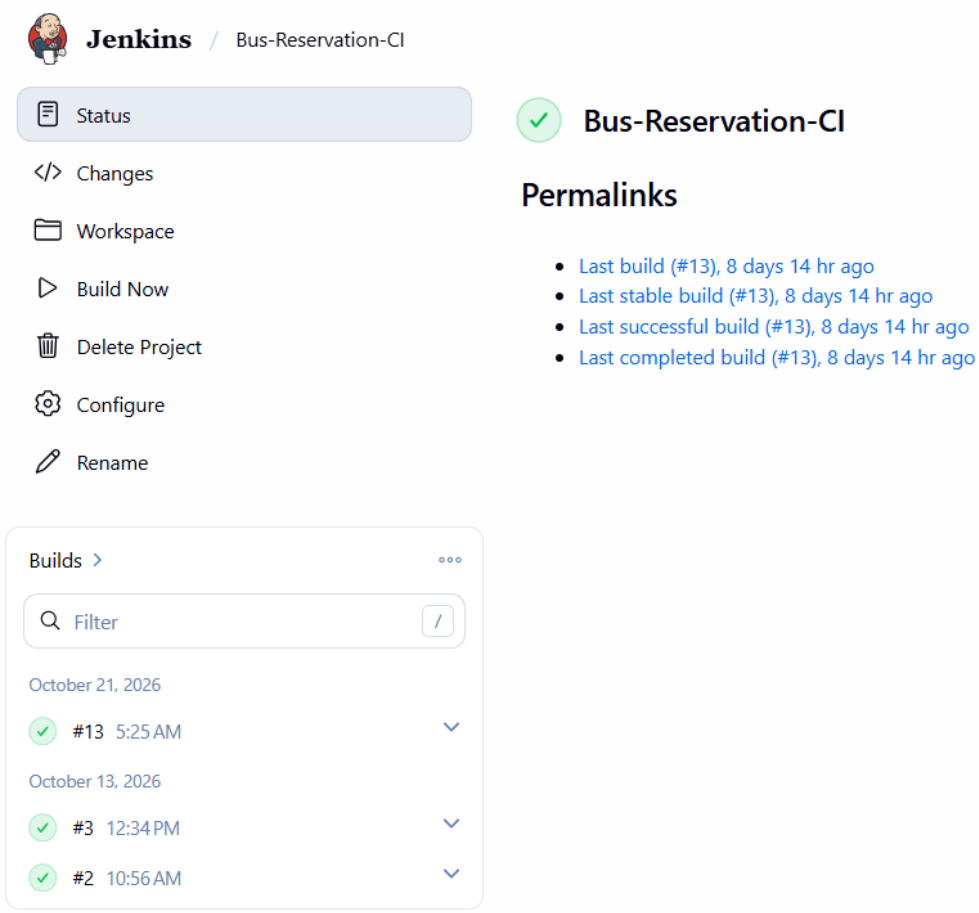
The ZIP contains a previously generated Surefire report for BusReservationApplicationTests with 5 tests, 0 failures and 0 errors. This is a stored report, not a new CI run. Capture a fresh Jenkins build number, console output and archived JAR to demonstrate current success.

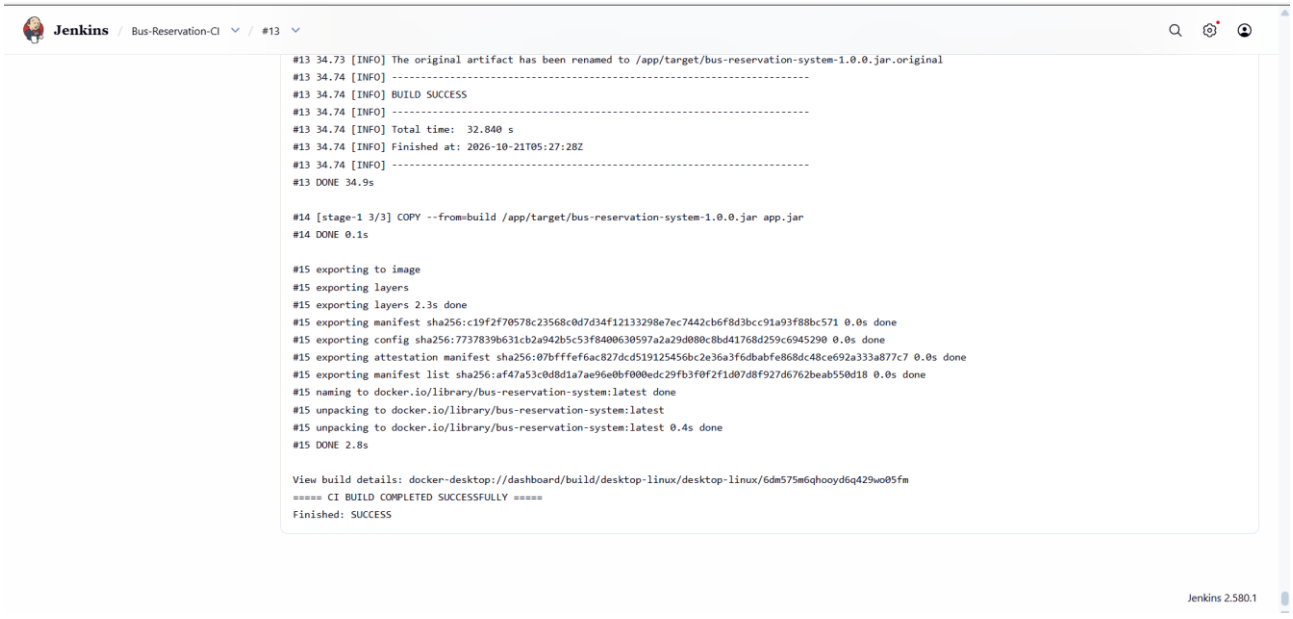
6. Troubleshooting

Symptom	Likely cause	Action
'bat' is not recognized / bat step failure	Pipeline is running on a non-Windows agent.	Run on a Windows agent or replace bat with sh commands appropriate to Linux.
Cannot read D:\maven-settings.xml	Path is missing or the Jenkins service account cannot access it.	Check file existence and permissions; use a valid or managed settings file.
mvn not found	Maven is not on the agent PATH or not configured as a tool.	Configure Maven in Jenkins Tools or update the agent environment.
Unsupported class version / wrong Java	Agent is using a JDK other than Java 21.	Set JAVA_HOME and PATH for the agent, then confirm java -version.

Tests fail	Application test failure or environment/configuration issue.	Read Surefire reports under target/surefire-reports and reproduce with mvn test.
No JAR archived	Packaging did not produce target/*.jar, or the build failed before packaging.	Inspect Package stage and target directory; consider tightening allowEmptyArchive after confirming expected artifact naming.

7. Evidence, Result and Conclusion





Result/status

Jenkins installation, job creation, trigger configuration and a successful run are not verified by the source ZIP alone. The repository does contain a Jenkinsfile for a Windows-based Maven CI flow. Complete the procedure and attach current screenshots before marking this task successful.

References: Jenkinsfile; pom.xml; README.md; src/main/resources/application.properties; Maven Surefire reports bundled in the source archive.

Task 08: Pipeline as Code and Server Deployment

DevOps Bus Reservation System

Student Name	DIKSHA PARULEKAR
Roll No.	23102A0068
Branch	CMPN A
Course	DevOps
Project	DevOps Bus Reservation System
Task	Task 08

1. Objective

Explain the repository's declarative Jenkins pipeline, how its build/test/package/artifact flow works, and what is required to deploy the resulting application to a server. Separate the current CI pipeline from deployment capabilities that are not implemented in the Jenkinsfile.

2. Pipeline-as-Code Concept

Pipeline as Code stores build automation alongside the application source in a version-controlled Jenkinsfile. This makes the stage sequence reviewable, repeatable and easier to audit than a job configured only through a graphical interface.

The repository's Jenkinsfile defines four stages—Checkout, Build, Test and Package—and a post-always artifact archive. It does not define a Deploy stage, server connection, service restart, health-check gate or rollback step. Therefore, this file is a CI pipeline with artifact archiving, not a complete server deployment pipeline.

3. Actual Jenkinsfile

```
pipeline {
    agent any

    environment {
        MAVEN_SETTINGS = 'D:\\maven-settings.xml'
    }

    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Build') {
            steps { bat "mvn -s ${env.MAVEN_SETTINGS} clean compile" }
        }
        stage('Test') {
            steps { bat "mvn -s ${env.MAVEN_SETTINGS} test" }
        }
        stage('Package') {
            steps { bat "mvn -s ${env.MAVEN_SETTINGS} package -DskipTests" }
        }
    }

    post {
        always {
            archiveArtifacts artifacts: 'target/*.jar', allowEmptyArchive: true
        }
    }
}
```

Element	What it means
agent any	Jenkins can allocate any eligible agent; however, bat commands require a Windows-compatible agent.
MAVEN_SETTINGS	A hard-coded Windows path that must exist and be readable by the agent account.
Checkout	Uses checkout scm to retrieve the revision configured for the Pipeline job.
Build	Runs clean compile using Maven and the configured settings file.
Test	Runs the root project's Maven tests.
Package	Creates package output while skipping tests because the Test stage already ran.
post / always	Attempts to archive target/*.jar after the pipeline, including failed runs; allowEmptyArchive prevents missing artifacts alone from failing this post action.

4. Build, Test and Artifact Flow

1. Jenkins checks out the configured repository revision.
2. Maven clean compile validates compilation and clears old build output.
3. Maven test executes tests configured in the root pom.xml, including the Spring Boot test class.
4. Maven package -DskipTests creates the packaged application without rerunning the tests.
5. Jenkins archives any matching target/*.jar files for later download or use by a separate release/deployment job.

The root pom.xml uses Spring Boot 3.2.4, Java 21, Spring Web, Data JPA, Security, Validation, H2 runtime and MySQL runtime dependencies. The application artifact name is bus-reservation-system-1.0.0.jar according to the README and Dockerfile.

5. Deployment Readiness and Procedure

Deployment prerequisites must be chosen for the actual target environment. The source repository does not identify a verified server address or deployment result. Before deploying, confirm the target OS, Java 21 runtime or container runtime, port/firewall policy, datasource settings, secrets, and an agreed application health endpoint.

6. Build and test the selected revision using Jenkins. Confirm the archived JAR exists and corresponds to that build.
7. Provision a test server or deployment environment under an approved account. Do not put SSH passwords, tokens or database passwords in source control.
8. Transfer the exact archived artifact through an approved mechanism and record its checksum/version.
9. Configure environment variables such as SPRING_DATASOURCE_URL and related datasource variables when using MySQL. The default configuration is H2 in-memory.
10. Start the application with the chosen runtime command and confirm the actual configured port. application.properties currently sets server.port=8085; the README quick-start and Dockerfile document port 8080, so verify or explicitly override the port mapping.
11. Perform a real smoke test of registration/login, bus listing, seat availability and a booking/cancellation cycle.
12. Only after a health check and smoke test pass should traffic be switched or the release marked successful. Keep a known previous artifact and documented rollback procedure.

The repository contains a Dockerfile and ansible/site.yml. The Dockerfile builds the JAR in a Maven/Java 21 stage and uses a Java 21 JRE runtime image; it declares EXPOSE 8080 and starts app.jar. The Ansible playbook targets an inventory group named webservers, creates /opt/bus_reservation, checks http://localhost:8080/actuator/health while ignoring errors, and prints a status message. These files are present, but their presence does not prove a server was provisioned or the health endpoint exists. The health-check task's ignore_errors: true means a failed health check does not necessarily fail the play.

6. Proposed Deployment Enhancement (Not Yet in Jenkinsfile)

```
// Illustrative stage only; adapt to the real target and credentials before use.
stage('Deploy') {
    steps {
        echo 'Deploy stage must be implemented for the selected environment'
```

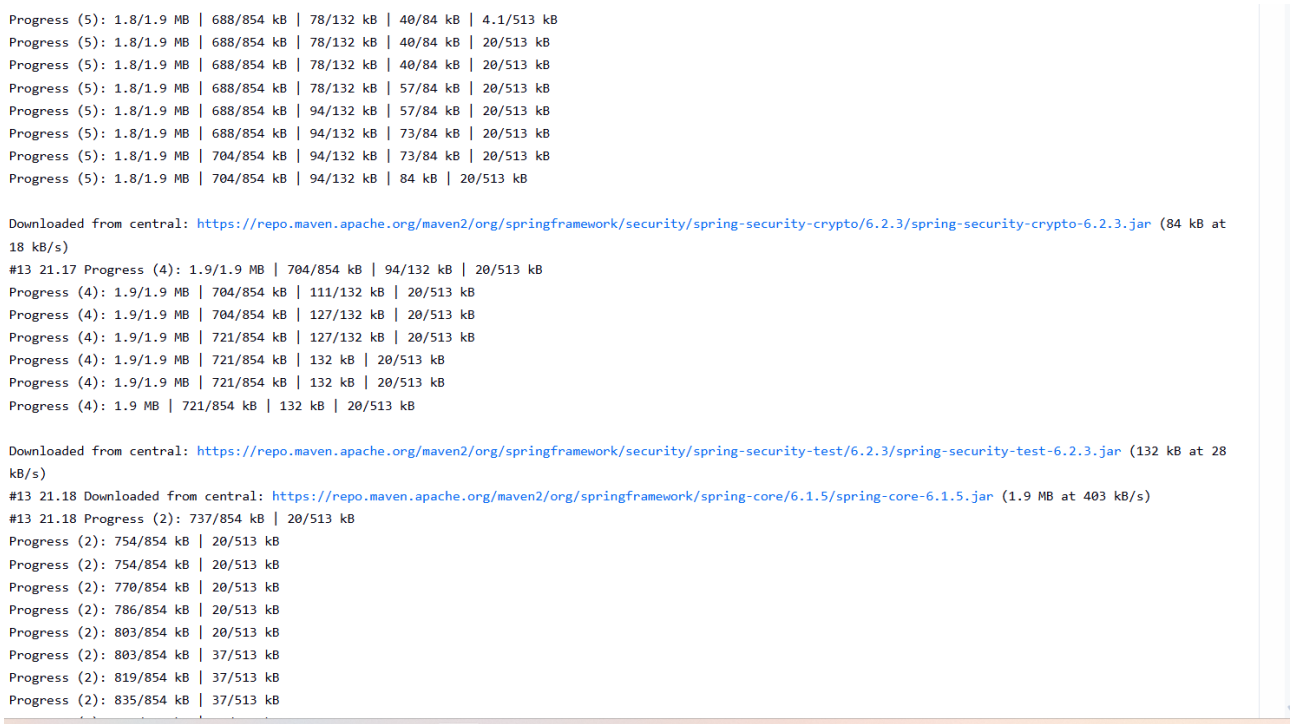
```
        // Add secure artifact transfer, service/container restart,
        // health check with failure handling, and rollback logic.
    }
}
```

The snippet above is explanatory, not a working deployment implementation. A production-ready stage should use Jenkins credentials, restrict deployment to approved branches/tags, fail on an unhealthy service, and preserve the previous working version. If using Docker, define image tagging, registry credentials, push/pull and container replacement. If using Ansible, configure a real inventory and remove or redesign ignored health-check failures.

7. Configuration and Security Considerations

- Replace the fixed D:\maven-settings.xml path with a documented managed configuration if the pipeline will run on multiple agents.
- Use a Windows agent for bat commands, or convert the pipeline to sh for Linux.
- Do not commit secrets or use shell command strings that expose tokens in logs.
- Resolve the port mismatch among application.properties (8085), README (8080) and Dockerfile (EXPOSE 8080) before deployment.
- Confirm /actuator/health is actually provided; the shown pom.xml does not explicitly declare Spring Boot Actuator.
- Add explicit deployment failure handling, health-check retries and rollback rather than relying on an echo message.

8. Evidence, Result and Conclusion



```
Progress (5): 1.8/1.9 MB | 688/854 kB | 78/132 kB | 40/84 kB | 4.1/513 kB
Progress (5): 1.8/1.9 MB | 688/854 kB | 78/132 kB | 40/84 kB | 20/513 kB
Progress (5): 1.8/1.9 MB | 688/854 kB | 78/132 kB | 40/84 kB | 20/513 kB
Progress (5): 1.8/1.9 MB | 688/854 kB | 78/132 kB | 57/84 kB | 20/513 kB
Progress (5): 1.8/1.9 MB | 688/854 kB | 94/132 kB | 57/84 kB | 20/513 kB
Progress (5): 1.8/1.9 MB | 688/854 kB | 94/132 kB | 73/84 kB | 20/513 kB
Progress (5): 1.8/1.9 MB | 704/854 kB | 94/132 kB | 73/84 kB | 20/513 kB
Progress (5): 1.8/1.9 MB | 704/854 kB | 94/132 kB | 84 kB | 20/513 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/security/spring-security-crypto/6.2.3/spring-security-crypto-6.2.3.jar (84 kB at
18 kB/s)
#13 21.17 Progress (4): 1.9/1.9 MB | 704/854 kB | 94/132 kB | 20/513 kB
Progress (4): 1.9/1.9 MB | 704/854 kB | 111/132 kB | 20/513 kB
Progress (4): 1.9/1.9 MB | 704/854 kB | 127/132 kB | 20/513 kB
Progress (4): 1.9/1.9 MB | 721/854 kB | 127/132 kB | 20/513 kB
Progress (4): 1.9/1.9 MB | 721/854 kB | 132 kB | 20/513 kB
Progress (4): 1.9/1.9 MB | 721/854 kB | 132 kB | 20/513 kB
Progress (4): 1.9 MB | 721/854 kB | 132 kB | 20/513 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/security/spring-security-test/6.2.3/spring-security-test-6.2.3.jar (132 kB at 28
kB/s)
#13 21.18 Downloaded from central: https://repo.maven.apache.org/maven2/org/springframework/spring-core/6.1.5/spring-core-6.1.5.jar (1.9 MB at 403 kB/s)
#13 21.18 Progress (2): 737/854 kB | 20/513 kB
Progress (2): 754/854 kB | 20/513 kB
Progress (2): 754/854 kB | 20/513 kB
Progress (2): 770/854 kB | 20/513 kB
Progress (2): 786/854 kB | 20/513 kB
Progress (2): 803/854 kB | 20/513 kB
Progress (2): 803/854 kB | 37/513 kB
Progress (2): 819/854 kB | 37/513 kB
Progress (2): 835/854 kB | 37/513 kB
```

```
#14 [stage-1 3/3] COPY --from=build /app/target/bus-reservation-system-1.0.0.jar app.jar
#14 DONE 0.1s

#15 exporting to image
#15 exporting layers
#15 exporting layers 2.3s done
#15 exporting manifest sha256:c19f2f70578c23568c0d7d34f12133298e7ec7442cb6f8d3bcc91a93f88bc571 0.0s done
#15 exporting config sha256:7737839b631cb2a942b5c53f8400630597a2a29d080c8bd41768d259c6945290 0.0s done
#15 exporting attestation manifest sha256:07bfffef6ac827dcd519125456bc2e36a3f6dbabfe868dc48ce692a333a877c7 0.0s
#15 exporting manifest list sha256:af47a53c0d8d1a7ae96e0bf000edc29fb3f0f2f1d07d8f927d6762beab550d18 0.0s done
#15 naming to docker.io/library/bus-reservation-system:latest done
#15 unpacking to docker.io/library/bus-reservation-system:latest
#15 unpacking to docker.io/library/bus-reservation-system:latest 0.4s done
#15 DONE 2.8s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/6dm575m6qhooyd6q429wo05fm
===== CI BUILD COMPLETED SUCCESSFULLY =====
Finished: SUCCESS
```

Result/status:

Pipeline-as-code and CI stages are implemented in the repository. A deployment stage is absent from the Jenkinsfile. Server deployment, health validation and rollback remain steps to configure and verify; no server or deployment success is claimed here.

References: Jenkinsfile; pom.xml; README.md; Dockerfile; ansible/site.yml; application.properties.

Task 09: Selenium Test Design and Local Execution

DevOps Bus Reservation System

Student Name	DIKSHA PARULEKAR
Roll No.	23102A0068
Branch	CMPN A
Course	DevOps
Project	DevOps Bus Reservation System
Task	Task 09

1. Objective

Inspect the Selenium end-to-end test project, describe its browser automation strategy and test flow, and document how to run it locally against the Bus Reservation System. Expected results are separated from results that have actually been verified.

2. Selenium Test Setup

The Selenium suite is a separate Maven project under selenium-tests/. Its pom.xml declares Selenium Java 4.18.1, JUnit Jupiter 5.10.2, WebDriverManager 5.7.0 and Java source/target 21. The test uses ChromeDriver and headless Chrome, with WebDriverManager setting up the driver.

Item	Observed source detail
Test class	selenium-tests/src/test/java/com/busreservation/selenium/ReservationSystemE2ETest.java
Framework	JUnit 5 annotations and assertions; @TestMethodOrder(MethodOrderer.OrderAnnotation.class).
Browser	ChromeDriver configured with ChromeOptions.
Headless settings	--headless=new, --remote-allow-origins=*, --no-sandbox, --disable-dev-shm-usage.
Wait strategy	WebDriverWait with a 10-second timeout and ExpectedConditions.
Base URL	http://localhost:8080 (constant BASE_URL in the source).
Driver cleanup	@AfterAll calls driver.quit() when the driver is not null.

3. Test Scenarios in Source

Order	Scenario	Source flow and assertion
1	User registration	Opens BASE_URL; fills regName, regEmail and regPassword; submits #registerForm; waits for alertBox and accepts a message containing “successful” or “already in use”.
2	User login	Fills loginEmail and loginPassword; submits #loginForm; waits for userNameDisplay and asserts it contains “Selenium Tester”.
3	Bus and seat selection	Waits for a clickable .bus-card, clicks it, waits for seatSection, clicks an AVAILABLE seat and checks bookSeatBtn is enabled.

The currently inspected Java source contains three @Test methods. The ZIP also contains a pre-existing Surefire text report recording seven tests, which does not match the three test methods visible in the source

file. Treat that report as a stale or mismatched build artifact until the suite is rebuilt. Do not report seven source tests based only on the old report.

4. Application Startup and Local Execution

The application.properties file sets server.port=8085 by default, while the Selenium test's BASE_URL is http://localhost:8080. Unless the port is overridden or the test URL is updated, the Selenium suite may target a different port from the app.

1. Open a terminal at the repository root BUS_RESERVATION_SYSTEM/BUS_RESERVATION_SYSTEM.
2. Confirm Java 21 and Maven are installed: java -version and mvn -version.
3. Build and test the application with mvn clean test. To start it locally, use mvn spring-boot:run or package and run the generated JAR.
4. Resolve the URL/port mismatch before running Selenium. Either configure the application to listen on 8080 for the test, or update BASE_URL to the actual app port 8085 and rebuild the Selenium project.
5. Keep the application running in one terminal. In a second terminal, enter selenium-tests/.
6. Run mvn test. Watch the output for driver setup, browser startup and test failures. Check selenium-tests/target/surefire-reports for the current test report.
7. After testing, stop the application and review the current report rather than relying on older files already present in target/.

```
# Terminal 1: application root
mvn clean test
mvn spring-boot:run

# Terminal 2: separate Selenium project
cd selenium-tests
mvn test
```

If the app is configured to use port 8085, either launch with a property override that sets server.port=8080 for this run, or change BASE_URL to http://localhost:8085 in the test source. Use one consistent choice and document it.

5. Selectors and Test Flow

Selector	Purpose
#regName	Registration name field.
#regEmail	Registration email field.
#regPassword	Registration password field.
#registerForm button[type='submit']	Registration submit button.
#alertBox	Registration feedback message.
#loginEmail / #loginPassword	Login credentials fields.
#loginForm button[type='submit']	Login submit button.
#userNameDisplay	Visible user identity after login.
.bus-card	Clickable bus card.
#seatSection	Seat selection section.
.AVAILABLE	A seat currently marked available.
#bookSeatBtn	Booking action button whose enabled state is asserted.

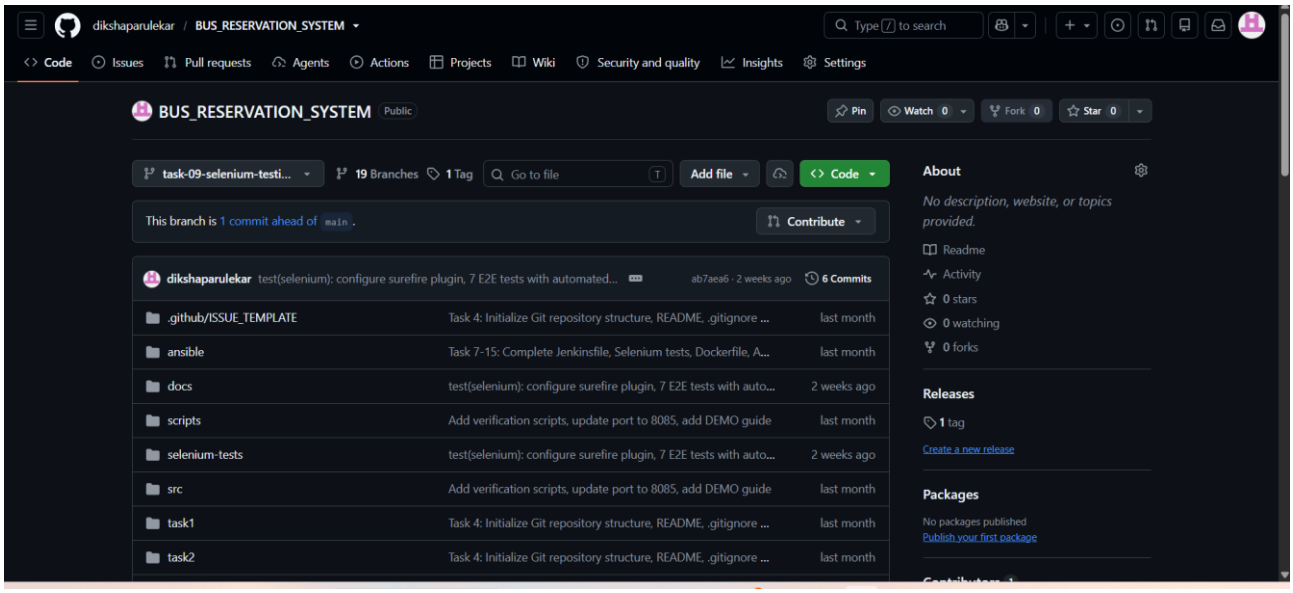
These selectors are literal values from the current test source. They must match the current frontend markup; if a UI refactor changes an ID or class, update the test and application consistently.

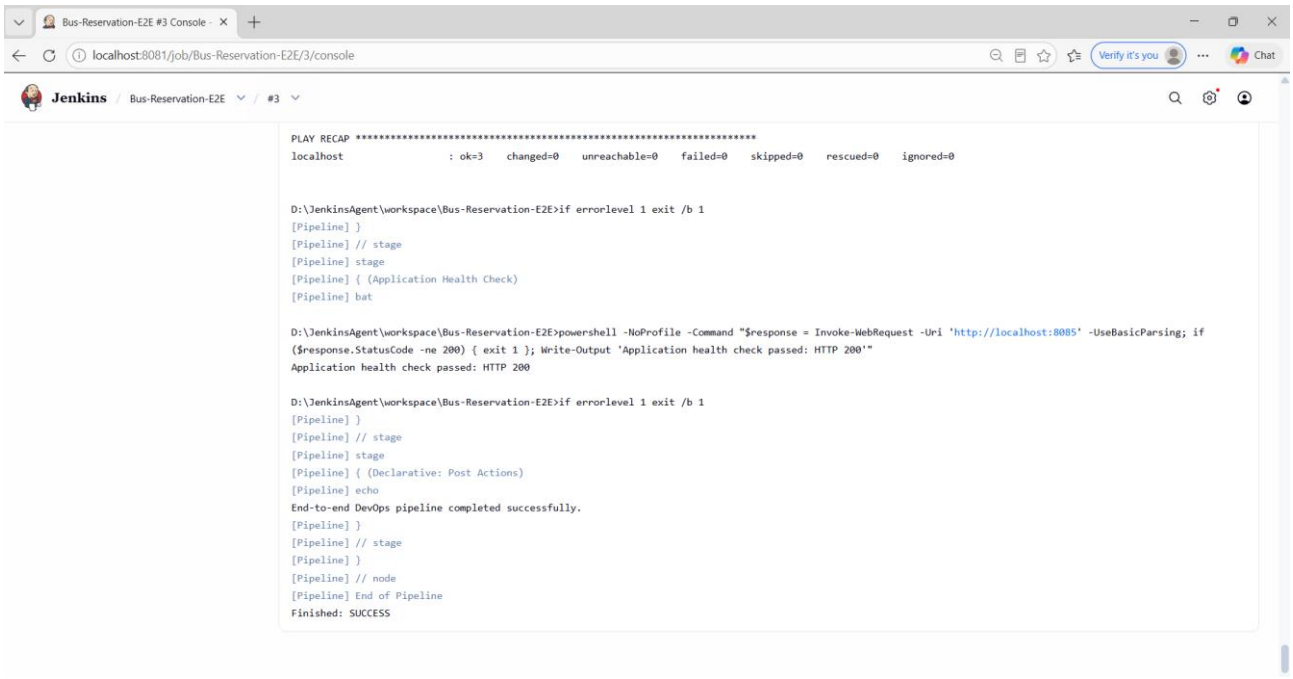
6. Expected Behaviour and Troubleshooting

Problem	Checks and corrective action
Connection refused / blank page	Start the app first; confirm the actual listening port and align it with BASE_URL.
NoSuchElementException / timeout	Inspect the page markup and selector spelling; wait for the relevant element and check

	whether login/registration errors changed the UI.
ChromeDriver startup failure	Confirm Chrome is installed and compatible; allow WebDriverManager network access or configure a compatible driver in the test environment.
Headless browser exits	Review Chrome crash logs, memory constraints and the --no-sandbox / --disable-dev-shm-usage options for the operating system.
Registration says already in use	The source explicitly accepts “already in use” for registration; ensure login data and expected user state remain consistent.
Later tests depend on previous state	The test class uses ordered methods and a shared static driver; check for state coupling and ensure test data is repeatable.

7. Evidence, Result and Conclusion





Result/status:

The Selenium project and test source are present. The source currently shows three ordered test methods and a localhost:8080 base URL; the application’s default port is 8085. A fresh successful run is not asserted. Align the port and execute the suite to establish current results.

References: selenium-tests/pom.xml; ReservationSystemE2ETest.java; root pom.xml; application.properties; bundled Surefire reports.

Task 10: Continuous Testing in Jenkins

DevOps Bus Reservation System

Student Name	DIKSHA PARULEKAR
Roll No.	23102A0068
Branch	CMPN A
Course	DevOps
Project	DevOps Bus Reservation System
Task	Task 10

1. Objective

Assess the existing automated application tests and Jenkins Test stage, then define the changes required to run both application tests and Selenium browser tests reliably in CI. The report does not assume Selenium is already integrated into Jenkins.

2. Existing Automated Tests

The root test class is src/test/java/com/busreservation/BusReservationApplicationTests.java. It uses @SpringBootTest and @Transactional, initializes sample bus data before each test, and exercises user registration/login, duplicate registration rejection, bus/seat availability, booking and duplicate-booking prevention, and cancellation.

Test area	Expected assertion/behaviour in source
Registration and login	Registered user receives an ID and can be authenticated by email/password.
Duplicate registration	A second registration for the same email throws a RuntimeException whose message contains “already in use”.
Bus and seat availability	The bus list and seat list are non-empty; first seat status is AVAILABLE.
Booking and duplicate booking	First booking has CONFIRMED status and seat status BOOKED; duplicate attempt throws an exception containing “already booked”.
Cancellation	Cancellation changes booking status to CANCELLED and seat status to AVAILABLE.

The project also has a separate selenium-tests Maven module. The current source file includes browser registration, login and bus/seat-selection tests, using JUnit 5, WebDriverManager, ChromeDriver and headless Chrome.

3. Actual Jenkins Test Stage

```
stage('Test') {
    steps {
        bat "mvn -s ${env.MAVEN_SETTINGS} test"
    }
}
```

This is the only test command in the checked-in Jenkinsfile. It executes Maven tests in the repository root according to the root pom.xml. The Selenium tests live in a separate Maven project with its own pom.xml; the existing Jenkinsfile does not change into selenium-tests/, does not run mvn test there, does not provision Chrome, and does not archive Selenium-specific reports or screenshots. Therefore, Selenium execution inside Jenkins is not established by the current configuration.

4. Integration Changes Required

Change	Why it is needed
Validate the Windows agent	Current pipeline uses bat and D:\maven-settings.xml. The selected agent must support both.
Run the application tests	Keep the existing root mvn test stage and confirm Surefire output is produced.
Build Selenium project separately	Add a stage that changes to selenium-tests and runs its Maven tests, or restructure Maven modules intentionally.
Provide a browser runtime	Install compatible Chrome on the Jenkins agent or use a controlled browser/container image; verify WebDriverManager can obtain the driver.
Start the application for E2E tests	Selenium requires the application to be reachable at BASE_URL. Start it as a background process or use a dedicated test environment and wait for readiness.
Fix URL/port mismatch	application.properties defaults to port 8085 while Selenium BASE_URL is localhost:8080.
Publish reports	Archive the root and Selenium surefire-reports outputs; configure JUnit report publishing if the Jenkins plugin is available.
Fail the pipeline on test failures	Do not mask the test exit code; make sure downstream packaging/deployment stages do not proceed after a failed test.

```
// Illustrative only: adapt to agent OS, paths, and application lifecycle.
stage('Selenium Tests') {
    steps {
        dir('selenium-tests') {
            bat 'mvn test'
        }
    }
}
post {
    always {
        junit allowEmptyResults: true, testResults: '**/target/surefire-reports/TEST-*.xml'
        archiveArtifacts artifacts: '**/target/surefire-reports/**, **/target/screenshots/**',
allowEmptyArchive: true
    }
}
```

The example shows the shape of an additional stage and report publication; it is not present in the current Jenkinsfile and is not ready to use without an application-startup step, port alignment, agent setup and path verification. In a production pipeline, prefer failing when expected reports are missing rather than leaving allowEmptyResults enabled indefinitely.

5. Test Reports and Failure Handling

1. Run the application tests locally with mvn test from the repository root and inspect target/surefire-reports.
2. Run Selenium locally only after starting the application and aligning BASE_URL with the configured server port.
3. In Jenkins, open Console Output and identify the first test failure, setup error or missing dependency.
4. Confirm XML reports are generated and that the Jenkins JUnit publisher points to the actual report paths.
5. After a failure is fixed, rerun the same revision and preserve both the failed run and the successful rerun as evidence.
6. Ensure the pipeline stops on failing tests before any future deployment stage. A package stage using -DskipTests is safe only when the earlier Test stage completed successfully.

Failure	Diagnostic
Maven test failure	Review target/surefire-reports/*.txt and TEST-*.xml; reproduce locally.
Selenium cannot connect	Confirm application startup, readiness, and localhost port 8080 versus configured 8085.
Chrome not found	Install/configure Chrome on the Jenkins agent or use a browser-capable container.
Driver download failure	Check network/proxy access for WebDriverManager or provide a controlled driver installation.
No report published	Check whether tests actually ran, verify report glob paths, and inspect whether reports are under root or selenium-tests.
Pipeline continues unexpectedly	Check stage order, test command exit status, and any catchError/allow-failure configuration added later.

6. Evidence, Result and Conclusion

```
Started by user Diksha Parulekar
[Pipeline] Start of Pipeline
[Pipeline] node
Still waiting to schedule task
‘windows-docker-agent’ is offline
Running on windows-docker-agent in D:\JenkinsAgent\workspace\Bus-Reservation-E2E
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Maven Build and Tests)
[Pipeline] bat

D:\JenkinsAgent\workspace\Bus-Reservation-E2E>set "JAVA_HOME=D:\Downloads\OpenJDK21U-jdk_x64_windows_hotspot_21.0.12.1_1\jdk-21.0.12.1+1"

D:\JenkinsAgent\workspace\Bus-Reservation-E2E>set "PATH=D:\Downloads\OpenJDK21U-jdk_x64_windows_hotspot_21.0.12.1_1\jdk-21.0.12.1+1\bin;D:\apache-maven-
3.10.0\bin;c:\Users\parul\AppData\Roaming\Code\User\globalStorage\github.copilot-
chat\debugCommand;c:\Users\parul\AppData\Roaming\Code\User\globalStorage\github.copilot-chat\copilotCli;C:\Program Files\Common
Files\Oracle\Java\javapath;C:\Program Files\Python312\Scripts\;C:\Program
Files\Python312\;C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\Wbem;C:\WINDOWS\System32\WindowsPowerShell\v1.0\;C:\WINDOWS\System32\OpenSSH\;C:\ProgramData\cho
colatey\bin;C:\Program Files\Java\jdk-23\bin;C:\Program Files\Git\cmd;C:\MinGW\bin;C:\Program
Files\MongoDB\Server\8.0\bin;C:\Users\parul\AppData\Roaming\Python\Python312\Scripts;C:\Program Files\MySQL\MySQL Server 8.0\bin;C:\Program
Files\Amazon\AWSCLIV2\;C:\Users\parul\AppData\Local\nvm;C:\nvm4w\nodejs;C:\Program Files\Docker\Docker\resources\bin;C:\Program Files\MySQL\MySQL Shell
8.0\bin\;C:\Users\parul\AppData\Local\Microsoft\WindowsApps;C:\Users\parul\AppData\Roaming\npm;C:\Program Files\Java\jdk-
17\bin;C:\MinGW\bin;C:\Users\parul\AppData\Local\Programs\mongosh\;C:\MinGW\bin;C:\Users\parul\AppData\Local\Google\Cloud SDK\google-cloud-
sdk\bin;C:\Users\parul\AppData\Local\nvm;C:\nvm4w\nodejs;C:\Users\parul\AppData\Local\GitHubDesktop\bin;D:\Kiro\bin;D:\Microsoft VS
Code\bin;C:\Users\parul\AppData\Local\Programs\Antigravity IDE\bin;D:\PythonPackages\Python312\Scripts;c:\Users\parul\.vscode\extensions\vscjava.vscode-java-debug-
0.59.0\bundled\scripts\noConfigScripts"

D:\JenkinsAgent\workspace\Bus-Reservation-E2E>set "MAVEN_OPTS=-Xms32m -Xmx256m"
```

```
D:\JenkinsAgent\workspace\Bus-Reservation-E2E>if errorlevel 1 exit /b 1
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Application Health Check)
[Pipeline] bat

D:\JenkinsAgent\workspace\Bus-Reservation-E2E>powershell -NoProfile -Command "$response = Invoke-WebRequest -Uri 'http://localhost:8085' -UseBasicParsing; if
($response.StatusCode -ne 200) { exit 1 }; Write-Output 'Application health check passed: HTTP 200'"
Application health check passed: HTTP 200

D:\JenkinsAgent\workspace\Bus-Reservation-E2E>if errorlevel 1 exit /b 1
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
End-to-end DevOps pipeline completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins 2.58

Result/status:

The repository contains a root application test class and a Jenkins Test stage that runs root Maven tests. Selenium tests are in a separate Maven project and are not invoked by the current Jenkinsfile. CI execution of Selenium, report publication and a test-failure quality gate require configuration and fresh evidence.

References: Jenkinsfile; pom.xml; BusReservationApplicationTests.java; selenium-tests/pom.xml; ReservationSystemE2ETest.java; application.properties.

TASK 11 Docker Image and Container Lifecycle

DevOps Bus Reservation System

Student Name	DIKSHA PARULEKAR
Roll Number	23102A0068

1. Objective

Build a reproducible container image for the Spring Boot bus reservation application and document safe image/container lifecycle operations.

2. Introduction

The supplied Dockerfile uses a multi-stage build: Maven with Eclipse Temurin 21 compiles/packages the application, then a Temurin 21 JRE Alpine image runs the JAR. No Docker Compose file was found in the inspected project paths. The application configuration sets server.port=8085 while the Dockerfile declares EXPOSE 8080; reconcile this before assuming a host-port mapping will work.

3. Technical Background and Architecture

Docker images package an application filesystem and runtime dependencies; containers are isolated processes created from those images. Image tags identify versions but should be paired with immutable digests for release traceability.

4. Project-Specific Implementation

Table 11.1 — Docker configuration/file inventory

File/setting	Observed configuration	Status
Dockerfile	Multi-stage Maven build; runtime JRE	Present
Build base image	maven:3.9.6-eclipse-temurin-21	Present
Runtime base image	eclipse-temurin:21-jre-alpine	Present
Build command	mvn -s settings.xml clean package -DskipTests	Present; tests skipped in image build
Artifact copy	target/bus-reservation-system-1.0.0.jar → app.jar	Present
Port	EXPOSE 8080; app config server.port=8085	Mismatch to resolve
Compose file	No compose YAML found in inspected paths	Not found

5. Implementation Notes

The Docker build context must include Dockerfile, pom.xml, settings.xml and src/. The Dockerfile copies settings.xml into the build stage and runs Maven packaging with tests skipped; tests should therefore be run separately before image publication. No Compose configuration was identified in the inspected project paths.

6. Diagram(s)

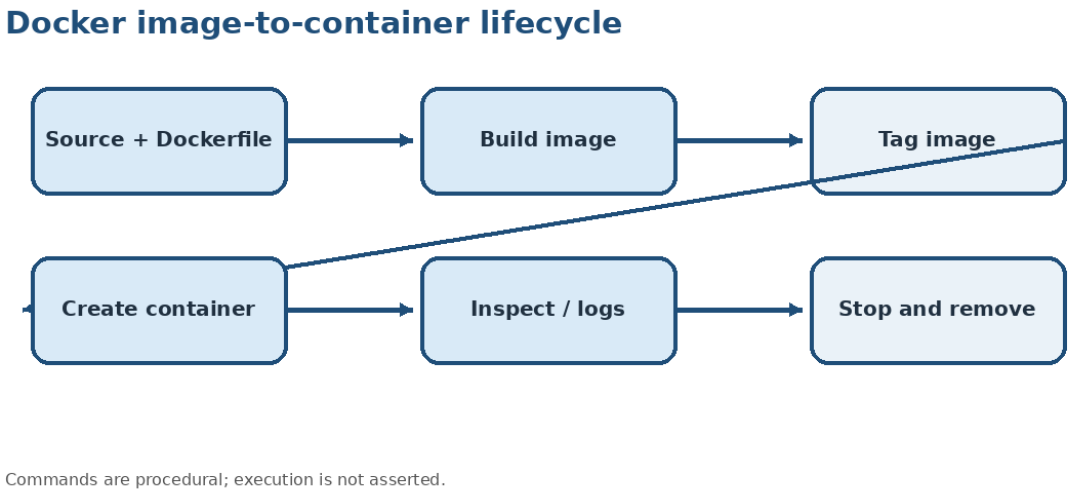


Figure 11.1 — Docker image build-to-container lifecycle

The sequence distinguishes image creation from container runtime operations. Each step should be verified using Docker CLI output.

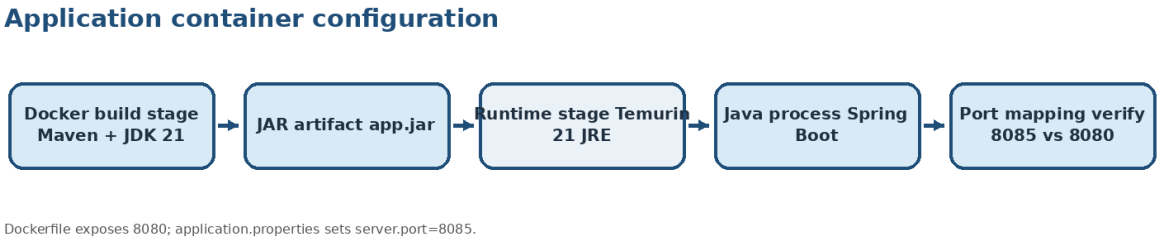


Figure 11.2 — Application components and container interaction

The image shows build and runtime layers. Important configuration finding: EXPOSE is metadata, not a port change; Spring Boot currently listens on 8085 unless overridden.

7. Configuration / Command Tables

Table 11.2 — Docker commands and purpose

Operation	Command	Purpose / caution
1	docker version	Confirm Docker client/engine availability
2	docker build -t bus-reservation:1.0.0 .	Build from project root containing Dockerfile
3	docker image ls bus-reservation	Confirm local image/tag
4	docker run -d --name bus-app -p 8085:8085 bus-reservation:1.0.0	Use only after confirming app listens on 8085; alternatively set server.port=8080 consistently
5	docker ps -a	Inspect container state and published ports
6	docker logs --tail 100 bus-app	Read startup and runtime logs
7	docker inspect bus-app	Inspect image, ports, env, mounts and state
8	docker stop bus-app; docker rm bus-app	Stop then remove container
9	docker image rm bus-reservation:1.0.0	Remove image only when no longer needed

8. Step-by-Step Procedure

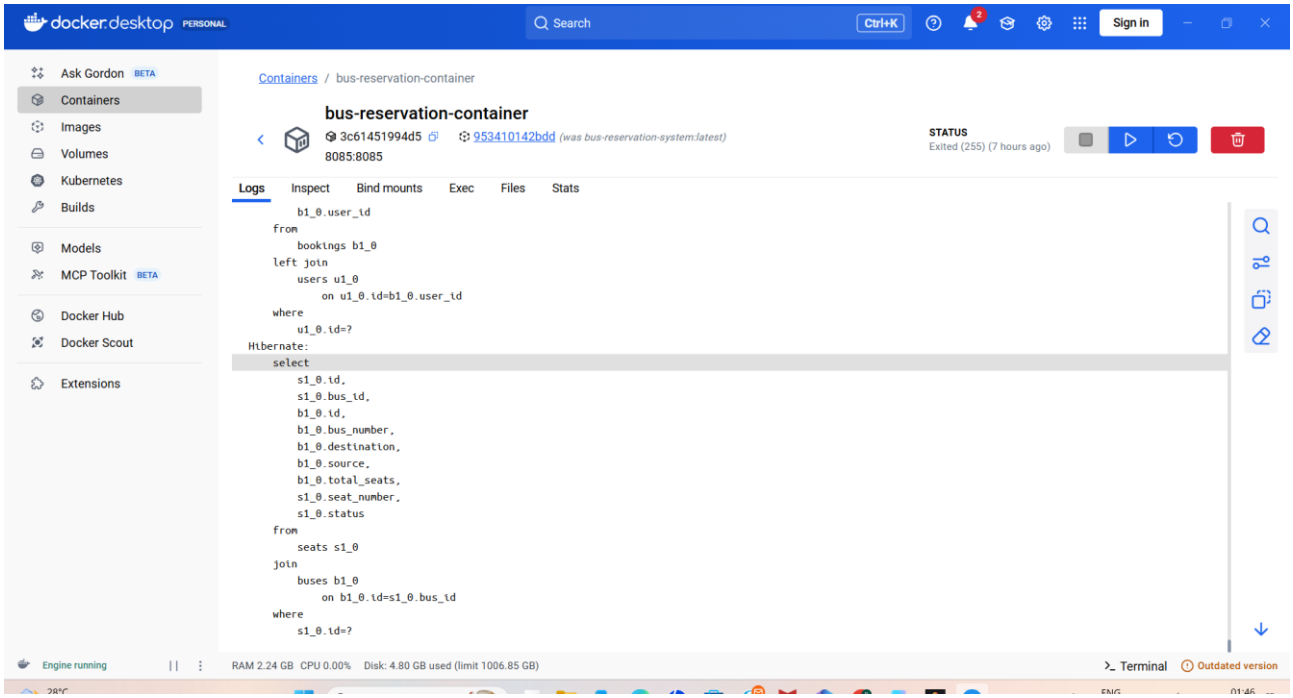
1. Open a terminal at the directory containing the supplied Dockerfile, pom.xml, settings.xml and src/.
2. Inspect Dockerfile and application.properties; decide one consistent application port before build/run.
3. Run `docker build -t bus-reservation:1.0.0 .` and preserve the complete build output.
4. Check the image with `docker image inspect bus-reservation:1.0.0`.
5. Start a test container with a port mapping matching the actual Spring Boot listening port. Do not assume EXPOSE changes server.port.
6. Check `docker ps`, `docker logs bus-app`, and `docker inspect bus-app`; test the UI/API endpoint in a browser or curl.
7. Capture evidence, then stop/remove the test container. Remove the image only if it is not needed.

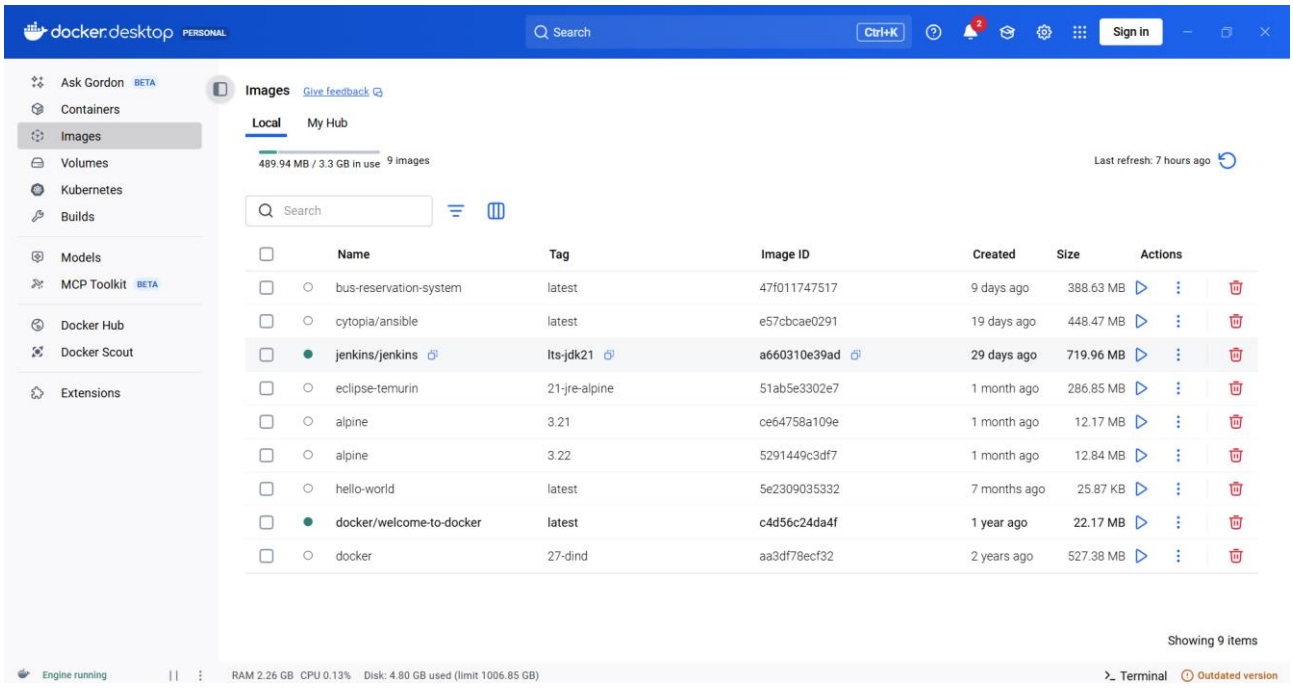
9. Testing, Validation and Task Status

Table 11.3 — Container lifecycle validation and troubleshooting

Validation	Command/check	Status
Docker daemon reachable	docker info	Pending local execution
Image builds	docker build ...	Not executed for this report
Container starts	docker ps -a + logs	Pending; port mismatch first
HTTP readiness	curl http://localhost:8085/actuator/health	Endpoint is referenced by Ansible, actuator dependency/config not confirmed
Cleanup	docker stop/rm	Execute only after evidence capture

10. Screenshot Placeholders and Evidence Instructions





11. Results and Implementation Status

Table 11.7 — Implementation status summary

Area	Observed status	Evidence/action needed
Dockerfile	Verified by source inspection	Build/run in local Docker and capture output
Compose configuration	No Compose file found in inspected paths	Use documented docker run or add Compose intentionally
Port alignment	Mismatch: 8080 EXPOSE vs 8085 app config	Choose one port and retest
Image/container execution	Not run as part of report generation	Capture genuine CLI output

12. Limitations and Troubleshooting

- If build cannot find settings.xml, check the Docker build context and COPY paths; settings.xml must be present beside Dockerfile.
- If the container starts then exits, inspect `docker logs bus-app` before changing the image.
- If the host receives connection refused, check actual server.port, published port, container state and logs.
- Do not use `docker system prune -a` casually; it may remove unrelated unused images and caches.

13. Conclusion

This report provides a project-specific, repeatable workflow while separating source-verified configuration from execution that still needs evidence. Final completion should be claimed only after the commands are run in the intended environment and current logs, screenshots, and test reports are retained.

14. References and Evidence

- Project `Dockerfile`.
- `src/main/resources/application.properties`.
- Project `pom.xml` and `docs/task11-docker-lifecycle.md`.
- Docker CLI documentation: https://docs.docker.com/reference/cli/docker/

TASK 12 REPORT

Jenkins-Docker Continuous Deployment

DevOps Bus Reservation System

Student Name	DIKSHA PARULEKAR
Roll Number	23102A0068

1. Objective

Evaluate the current Jenkins pipeline and define an evidence-based path from CI artifact creation to Docker-based deployment without misrepresenting proposed stages as implemented.

2. Introduction

The actual Jenkinsfile has Checkout, Build, Test and Package stages and archives `target/*.jar` in an always post-action. It uses a Windows `bat` step and `D:\maven-settings.xml`. It does not currently build a Docker image, authenticate to a registry, deploy a container, or run a deployment health check.

3. Technical Background and Architecture

Jenkins Pipeline as Code makes CI stages reviewable alongside source. Continuous deployment requires additional image publication, target environment, secret handling, health validation and rollback controls.

4. Project-Specific Implementation

Table 12.1 — Existing versus proposed pipeline stages

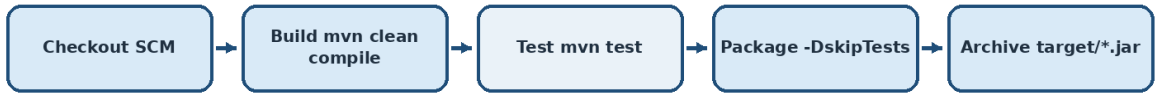
Stage	In Jenkinsfile ?	Evidence/configuration	Notes
Checkout	Yes	checkout scm	Confirm SCM job configuration
Build	Yes	mvn clean compile	Requires Windows-compatible agent and settings file
Unit/integration test	Yes	mvn test	Inspect Surefire reports/build status
Package JAR	Yes	mvn package -DskipTests	Tests already run in prior stage
Build image	No	docker build -t ...	Proposed stage; Docker engine needed
Registry login/push	No	credential-bound login + push	Proposed; store secret in Jenkins credentials
Deploy container	No	remote/agent deployment	Proposed; define target, rollback and secrets
Health verification	No	HTTP probe / health endpoint	Proposed; endpoint and actuator must be confirmed

5. Implementation Notes

The environment variable MAVEN_SETTINGS is set to a Windows path and pipeline commands use `bat`. A Linux-only agent will not execute those steps as written. The post/always block archives target/*.jar even after failure and allows an empty archive, so archive presence and build status must be checked separately.

6. Diagram(s)

Jenkinsfile pipeline currently present



Docker build, registry push and deployment are not in the supplied Jenkinsfile.

Figure 12.1 — Jenkins pipeline stages and artifact flow

Stages shown are the ones present in the checked-in Jenkinsfile. The archive action is configured in post/always, with allowEmptyArchive enabled.

Jenkins-to-Docker deployment target (proposed)



Only checkout/build/test/package/archive are present in Jenkinsfile.

Figure 12.2 — Jenkins-to-Docker deployment architecture (proposed)

Docker build, registry, deployment host, health check and rollback are future pipeline components, not current Jenkinsfile stages.

7. Configuration / Command Tables

Table 12.2 — Docker image lifecycle commands/configuration

Action	Command/template	Purpose / status
Build image	docker build -t bus-reservation:%BUILD_NUMBER% .	Proposed Windows batch example; ensure Dockerfile port alignment
Tag image	docker tag bus-reservation:%BUILD_NUMBER% registry.example/bus-reservation:%BUILD_NUMBER%	Replace registry path; do not commit secrets
Push image	docker push registry.example/bus-reservation:%BUILD_NUMBER%	Proposed; authenticate with Jenkins credential binding
Inspect image	docker image inspect bus-reservation:%BUILD_NUMBER%	Local image metadata validation
Deployment check	docker ps -a; docker logs bus-app	Validate actual target host after deployment

8. Step-by-Step Procedure

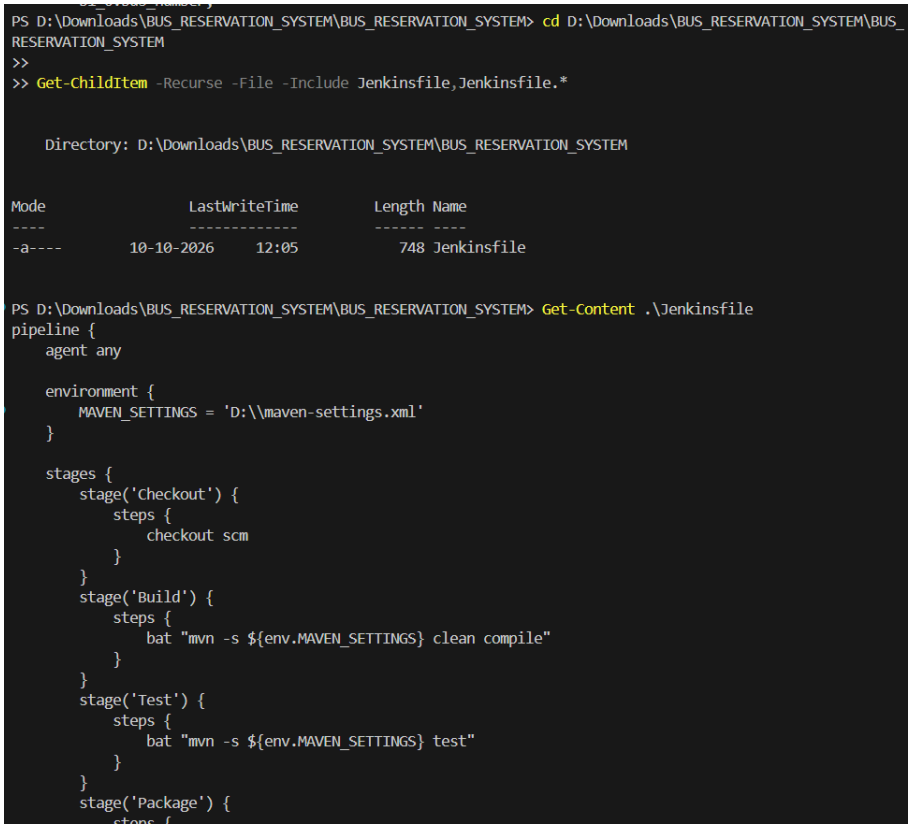
1. Configure a Jenkins Pipeline job that checks out the repository and uses the checked-in Jenkinsfile.
2. Ensure the agent is Windows-capable because the pipeline uses `bat`, and verify the Maven settings path exists on that agent.
3. Run a build and review Checkout, Build, Test and Package stage logs.
4. Confirm the archived JAR is present and identify its version/hash.
5. For a future Docker stage, align server.port and Docker port settings; install/configure Docker on the agent.
6. Add registry credentials through Jenkins Credentials and credential binding; never hard-code passwords or tokens.
7. Add deployment only after defining target host, image-tag policy, health endpoint, rollback and permissions. Validate with a test environment first.

9. Testing, Validation and Task Status

Table 12.3 — Deployment prerequisites and validation checklist

Check	Evidence/action	Status
Jenkinsfile syntax	Review declarative pipeline structure	Source inspected; no Jenkins run performed
Build/test/package	Run Jenkins build and inspect stage results	Historical source archive has Maven reports; not evidence of current Jenkins run
Artifact archive	Check archived target JAR	Configured, actual build not verified
Docker image stage	Not present	Proposed
Deployment and health	Not present	Proposed

10. Screenshot Placeholders and Evidence Instructions



```
PS D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM> cd D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_
RESERVATION_SYSTEM
>>
>> Get-ChildItem -Recurse -File -Include Jenkinsfile,Jenkinsfile.*


    Directory: D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10-10-2026     12:05            748 Jenkinsfile


PS D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM> Get-Content .\Jenkinsfile
pipeline {
    agent any

    environment {
        MAVEN_SETTINGS = 'D:\\maven-settings.xml'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }
        stage('Build') {
            steps {
                bat "mvn -s ${env.MAVEN_SETTINGS} clean compile"
            }
        }
        stage('Test') {
            steps {
                bat "mvn -s ${env.MAVEN_SETTINGS} test"
            }
        }
        stage('Package') {
```

```
PS D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM> Get-ChildItem .\target\*.jar


    Directory: D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM\target


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        01-02-1980     05:30       53990994 bus-reservation-system-1.0.0.jar


PS D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM> Select-String -Path .\Jenkins
"archiveArtifacts"

Jenkinsfile:33:            archiveArtifacts artifacts: 'target/*.jar', allowEmptyArchive: tr


PS D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM> ^C
PS D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM> docker --version
>> docker info
Docker version 29.1.3, build f52814d
Client:
 Version:    29.1.3
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.17.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.30.1-desktop.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.0.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
```

```
PS D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM> Get-ChildItem -Recurse -File -Filter Docker
file


    Directory: D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        14-10-2026     15:55            411 Dockerfile


PS D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM> Select-String -Path .\Jenkinsfile -Pattern
"stage\s*\(|docker|build -t|docker build"

Jenkinsfile:9:        stage('Checkout') {
Jenkinsfile:14:        stage('Build') {
Jenkinsfile:19:        stage('Test') {
Jenkinsfile:24:        stage('Package') {


PS D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM> docker images
                                                                                  i Info →  U  In Use
IMAGE                                ID             DISK USAGE   CONTENT SIZE   EXTRA
alpine:3.21                          ce64758a109e       12.2MB         3.71MB
alpine:3.22                          5291449c3df7       12.8MB         3.88MB
bus-reservation-system:latest        47f011747517        389MB          123MB
cytopia/ansible:latest               e57cbcae0291        448MB         68.2MB
docker/welcome-to-docker:latest      c4d56c24da4f       22.2MB         6.03MB    U
docker:27-dind                       aa3df78ecf32        527MB          137MB
eclipse-temurin:21-jre-alpine        51ab5e3302e7        287MB           75MB
hello-world:latest                   5e2309035332       25.9kB         9.49kB
jenkins/jenkins:lts-jdk21            a660310e39ad        720MB          246MB    U
PS D:\Downloads\BUS_RESERVATION_SYSTEM\BUS_RESERVATION_SYSTEM>
```

11. Results and Implementation Status

Table 12.7 — Implementation status summary

Area	Observed status	Evidence/action needed
Checkout/build/test/package	Present in Jenkinsfile	Run pipeline and save build URL/logs
Artifact archive	Configured in post/always	Verify archive exists for a successful build
Docker build and registry	Absent from Jenkinsfile	Implement only if required by deployment design
Deployment/rollback	Absent from Jenkinsfile	Document target, health check, and recovery plan

12. Limitations and Troubleshooting

- If `bat` fails on a Linux agent, use a Windows agent or deliberately port commands to `sh`.
- If Maven cannot read `D:\maven-settings.xml`, configure a valid agent-local settings path and avoid machine-specific assumptions.
- If archive is empty, check package output path and whether the build reached Package; allowEmptyArchive may hide absence of artifact.
- Use immutable build tags and Jenkins-managed credentials; never print secrets into console logs.

13. Conclusion

This report provides a project-specific, repeatable workflow while separating source-verified configuration from execution that still needs evidence. Final completion should be claimed only after the commands are run in the intended environment and current logs, screenshots, and test reports are retained.

14. References and Evidence

- Project `Jenkinsfile`.
- Project `Dockerfile` and `pom.xml`.
- `docs/task07-jenkins-ci.md`.
- Jenkins Pipeline documentation: https://www.jenkins.io/doc/book/pipeline/
- Docker CLI documentation: https://docs.docker.com/reference/cli/docker/

TASK 13 REPORT

Configuration Management Script

DevOps Bus Reservation System

Student Name	DIKSHA PARULEKAR
Roll Number	23102A0068

1. Objective

Inspect the existing Ansible inventory and playbook, explain their current behavior, and provide a safe workflow for configuration validation.

2. Introduction

The repository contains `ansible/inventory.ini` with a local `webservers` target and `ansible/site.yml`. The playbook ensures `/opt/bus_reservation` exists, calls `http://localhost:8080/actuator/health` with expected status 200 while ignoring errors, and prints a debug status message. Because errors are ignored, the debug message alone does not prove that the application is healthy. The endpoint/port and actuator availability require validation.

3. Technical Background and Architecture

Ansible uses an inventory to identify targets and a playbook to describe desired state. Idempotency and explicit failure handling are essential to safe configuration management.

4. Project-Specific Implementation

Table 13.1 — Ansible/script/documentation inventory

Artifact	Observed content	Status
ansible/inventory.ini	[webservers] localhost ansible_connection=local	Present
ansible/site.yml target	hosts: webservers	Present
Directory task	/opt/bus_reservation state=directory mode 0755	Present
Health URI	http://localhost:8080/actuator/health; status_code 200	Present; port/Actuator need confirmation
Error handling	ignore_errors: true on URI task	Present; may allow play to continue despite failed health check
Variables	No explicit vars block discovered in this playbook	None confirmed
Additional scripts	No separate config script verified in inspected artifact list	Not confirmed

Table 13.2 — Supported configuration variables/values

Variable or setting	Observed value	Purpose / caveat
Inventory group	webservers	Target group referenced by playbook
Connection	ansible_connection=local	Runs tasks on local machine; Linux path assumption
Directory path	/opt/bus_reservation	Directory to create
Directory mode	0755	Owner write; group/others read+execute
Health URL	http://localhost:8080/actuator/health	Expected status 200; confirm actual port and Actuator dependency

5. Implementation Notes

The playbook currently describes only a directory task and URI probe. It does not install Java/Docker, configure a service, deploy the JAR, or define rollback. The health probe references port 8080 while the application defaults to 8085, and ignore_errors can mask failure.

6. Diagram(s)



Figure 13.1 — Configuration-management workflow using actual Ansible artifacts

This diagram follows the inventory and tasks in the checked-in files. The health request failure is tolerated, so task recap and actual URI result must be inspected.

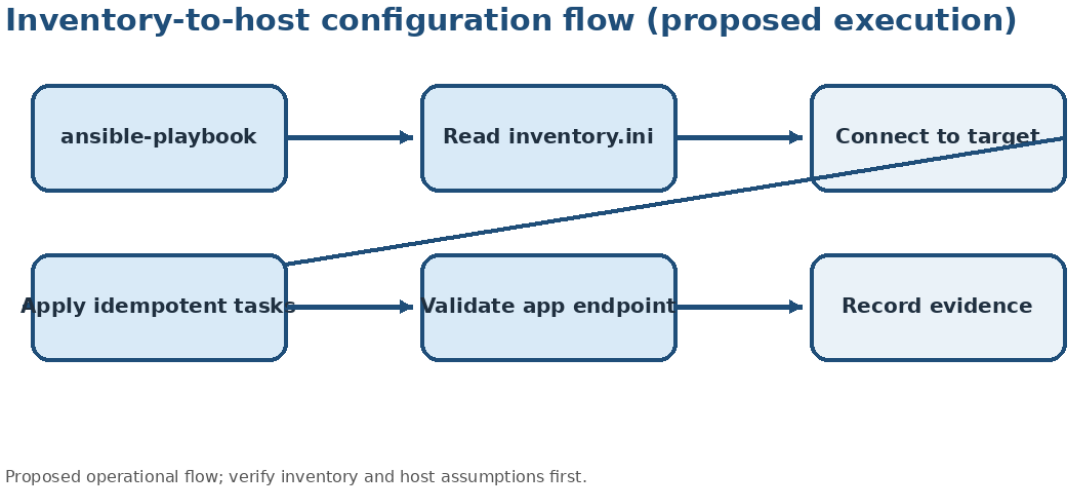


Figure 13.2 — Inventory-to-host configuration flow (proposed execution)

This flow describes how to execute and verify Ansible against an inventory. It is a proposed operational procedure, not proof that provisioning has succeeded.

7. Configuration / Command Tables

Table 13.3 — Configuration execution commands

Action	Command	Purpose
Syntax check	ansible-playbook -i ansible/inventory.ini ansible/site.yml --syntax-check	Checks playbook syntax only
Dry run	ansible-playbook -i ansible/inventory.ini ansible/site.yml --check	May not predict every module change
Run playbook	ansible-playbook -i ansible/inventory.ini ansible/site.yml	Execute only on a suitable host; /opt is Linux-specific
Verbose run	ansible-playbook -i ansible/inventory.ini ansible/site.yml -vv	Inspect task-level output
HTTP probe	curl -i http://localhost:8080/actuator/health	Use only after verifying endpoint and service port

8. Step-by-Step Procedure

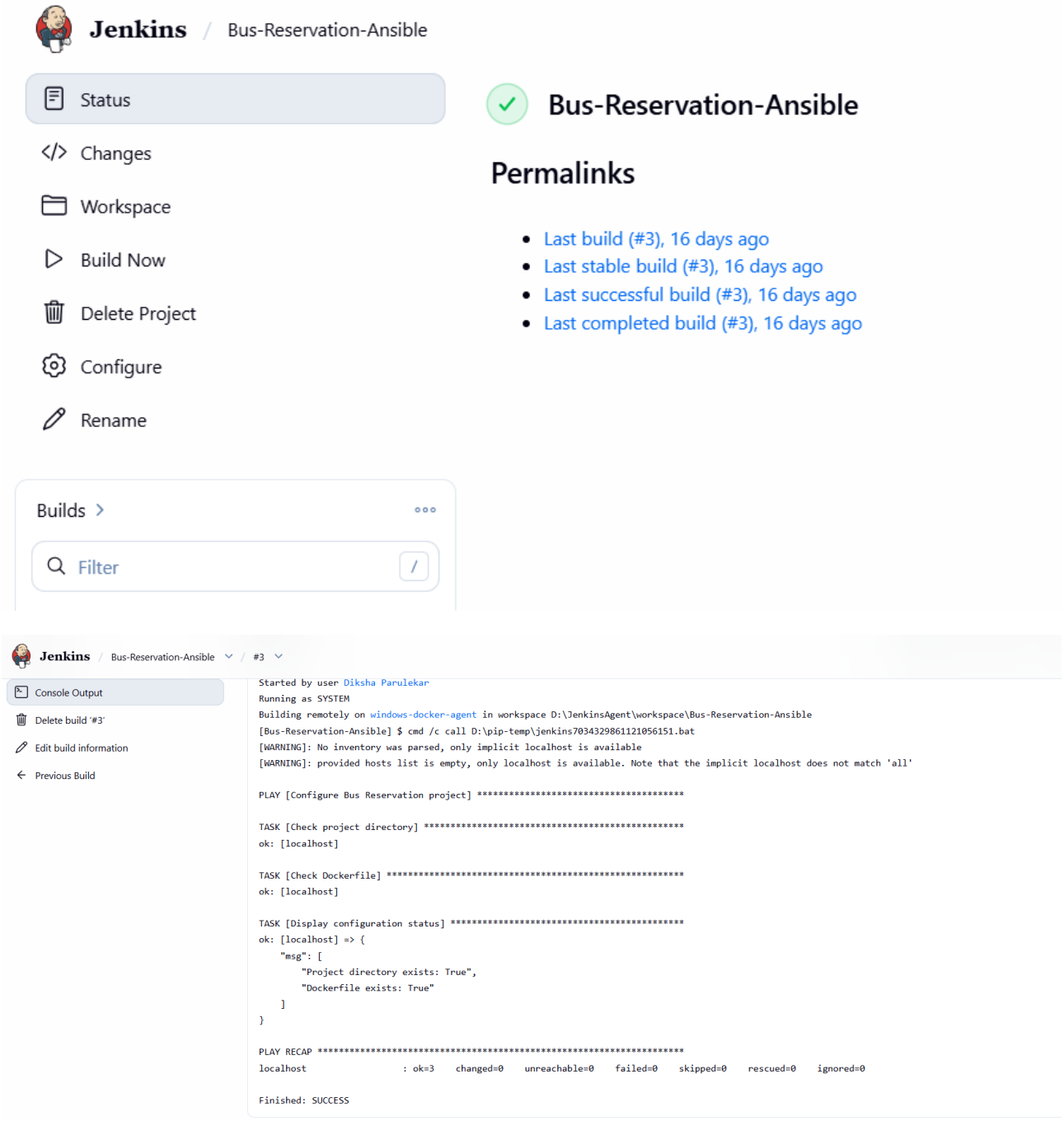
1. Confirm Ansible is installed on the control machine and that the target is Linux-compatible with `/opt` paths.
2. Read `ansible/inventory.ini` and confirm the intended target; current inventory uses a local connection.
3. Run the syntax check and inventory listing before making changes.
4. Use `--check` where supported, then execute against a disposable/test environment.
5. Inspect the URI task result independently. Because `ignore_errors: true` is present, do not treat a later debug message as proof of health.
6. Verify directory ownership/mode and probe the actual application endpoint/port.
7. Record play recap, command, target, and timestamps; remove or redact sensitive environment information.

9. Testing, Validation and Task Status

Table 13.4 — Configuration execution and validation checklist

Check	Validation method	Status
Inventory parsing	ansible-inventory -i ansible/inventory.ini --list	Pending local execution
Syntax	--syntax-check	Pending local execution
Directory task	Check /opt/bus_reservation on target	Linux/local target required
Health check	URI result must show HTTP 200 and healthy payload	Current playbook ignores URI task errors; strengthen before relying on it
Idempotency	Run twice and compare changed counts	Not verified

10. Screenshot Placeholders and Evidence Instructions



11. Results and Implementation Status

Table 13.7 — Implementation status summary

Area	Observed status	Evidence/action needed
Inventory and playbook	Verified by source inspection	Run syntax check and capture recap
Directory creation	Task is defined, not executed for this report	Verify path and permissions on target
Health validation	URI task defined; errors ignored	Confirm endpoint/port; make failure fatal if required
Variables/templates/roles	No explicit variables/templates/roles confirmed in inspected playbook	Add only if configuration scope requires them

12. Limitations and Troubleshooting

- If `/opt/bus_reservation` fails on Windows, run against a Linux host or use a platform-appropriate path and tasks.
- If the health URI fails, check whether the app is running, whether the port is correct, and whether Actuator is included and exposed.
- Consider removing `ignore_errors: true` or adding an explicit failed_when/assertion so failed health checks stop the play.
- Use least-privilege inventory credentials for remote hosts; do not place passwords in inventory files.

13. Conclusion

This report provides a project-specific, repeatable workflow while separating source-verified configuration from execution that still needs evidence. Final completion should be claimed only after the commands are run in the intended environment and current logs, screenshots, and test reports are retained.

14. References and Evidence

- `ansible/inventory.ini`.
- `ansible/site.yml`.
- `src/main/resources/application.properties`.
- Ansible playbook guide: https://docs.ansible.com/ansible/latest/playbook_guide/playbooks_intro.html

TASK 14 REPORT

Automated Provisioning and Reliability Validation

DevOps Bus Reservation System

Student Name	DIKSHA PARULEKAR
Roll Number	23102A0068

1. Objective

Define repeatable provisioning and reliability checks using the actual Maven/Spring test setup and Selenium test project, while separating archived test evidence from a fresh validation run.

2. Introduction

The source includes a Spring Boot test class and a Selenium test project. Archived Surefire text reports show 5 application tests and 7 Selenium tests with zero failures/errors in those recorded runs. These are historical files inside the supplied ZIP, not a new run performed while generating this report. The Selenium archive also contains JVM crash/dump logs, so repeat tests in a controlled environment and review current output.

3. Technical Background and Architecture

Provisioning is successful only when dependencies, application startup, database connectivity and user journeys are validated. Repeatable tests and retained output make reliability claims auditable.

4. Project-Specific Implementation

Table 14.1 — Prerequisites and environment configuration

Component	Observed configuration	Validation note
Java	Java 21 declared in pom.xml	Prerequisite
Spring Boot	3.2.4 parent version	Verified in pom.xml
Build	Maven; executable JAR target/bus-reservation-system-1.0.0.jar	Configured
Default database	H2 in-memory; environment-variable override for JDBC URL	Configured
Alternate driver	MySQL connector dependency	Available; active deployment config not proven
Application port	server.port=8085	Configured
Selenium	JUnit/Selenium Maven project in selenium-tests/	Present
Health check	Ansible references /actuator/health on port 8080	Endpoint availability not confirmed

5. Implementation Notes

The ZIP contains historical Surefire summaries: the application suite records 5 tests and the Selenium suite records 7 tests, each with zero failures/errors in those particular reports. Several Selenium JVM crash/dump files are also present. Rerun tests and investigate dumps before drawing current reliability conclusions.

6. Diagram(s)

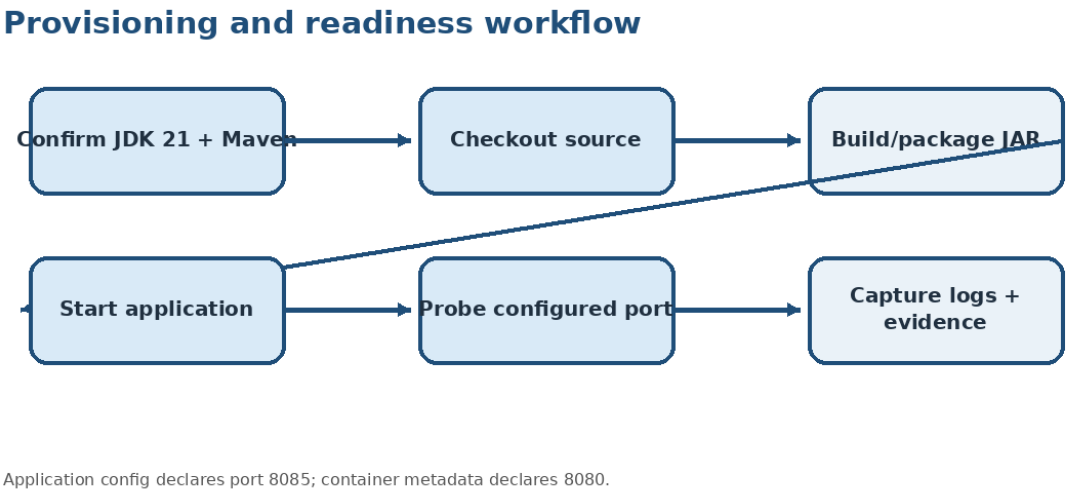


Figure 14.1 — Environment provisioning and application-readiness workflow

The workflow moves from prerequisites to packaged artifact and endpoint validation. The project config declares port 8085; Docker metadata and Ansible health URL use 8080 and need alignment.

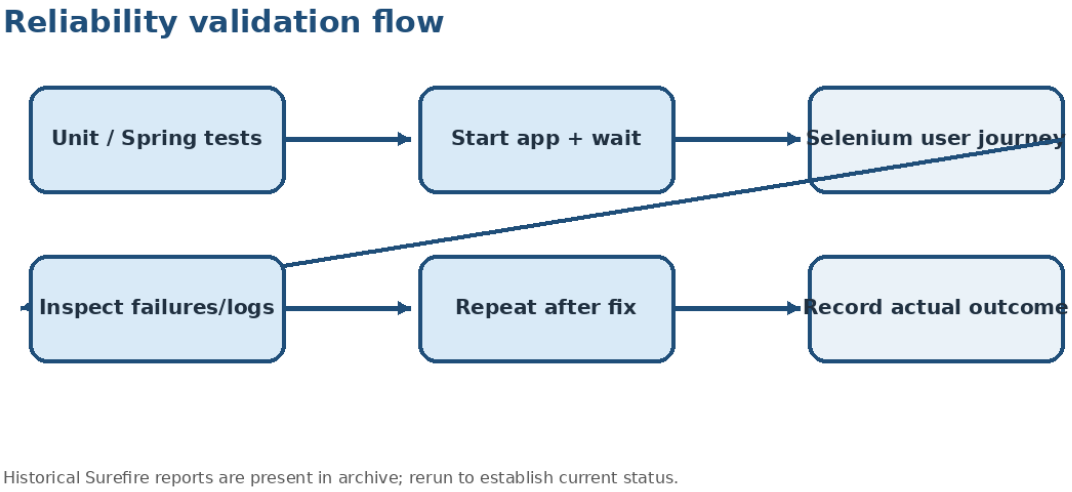


Figure 14.2 — Testing and reliability-validation flow

Unit/application tests and Selenium user journeys should be rerun and their actual reports retained. Historical report summaries are not a substitute for a current run.

7. Configuration / Command Tables

Table 14.2 — Test and validation cases

ID	Test	Command/action	Expected outcome	Evidence status
A01	Maven compile/package	mvn clean package	Exit code 0 and JAR exists	Execute current run
A02	Spring application tests	mvn test	Surefire shows tests/failures/errors	Archived report: 5 run, 0 failures/errors; rerun
A03	Selenium E2E	mvn test -f selenium-tests/pom.xml	Browser journeys complete and report generated	Archived report: 7 run, 0 failures/errors; rerun
A04	Startup readiness	Run JAR and probe correct port	HTTP response matches configured endpoint	Pending current run

ID	Test	Command/action	Expected outcome	Evidence status
A05	Database connectivity	Inspect logs and perform read/write workflow	No connection/schema exceptions	Pending current run
A06	Repeatability	Repeat build/test after clean checkout	Consistent result and retained reports	Pending current run
A07	Failure handling	Stop service or invalid endpoint in test environment	Failure is detected and reported clearly	Proposed validation

8. Step-by-Step Procedure

1. Verify JDK 21 and Maven versions and inspect project settings.
2. From the application root, run `mvn clean test`; review the Surefire report and console output.
3. Run `mvn package` and verify the expected JAR exists.
4. Start the application with the intended database profile/configuration; confirm its actual listening port (default 8085 from properties).
5. Run Selenium tests with the application ready and configured browser/driver. Use the Selenium project POM and retain current Surefire output.
6. Repeat tests from a clean checkout or clean build to check repeatability.
7. Document exact environment, command, test counts, failures, logs, and evidence; distinguish historical reports from current runs.

9. Testing, Validation and Task Status

Table 14.3 — Failure symptoms, causes and troubleshooting

Symptom	Possible cause	Troubleshooting
Maven dependency/build failure	JDK/Maven mismatch, network, settings.xml	Check `java -version`, `mvn -version`, Maven settings and full build log
Application does not start	Port occupied, config error, DB issue	Inspect console logs; verify port 8085 and datasource settings
Health probe fails	Wrong port or Actuator unavailable	Confirm dependency, exposure and URL; do not ignore failed probe
Selenium timeout	App not ready, wrong URL, browser/driver mismatch	Start app first, add explicit readiness wait, check browser logs
JVM crash dump	Environment/resource/native browser issue	Review dump; reduce parallelism, align browser/driver and rerun
Flaky reservation test	Shared state/timing/test data collision	Use isolated test data, deterministic setup and cleanup

10. Results and Implementation Status

Table 14.7 — Implementation status summary

Area	Observed status	Evidence/action needed
Spring test class	Present; archived report has 5/5 with no recorded failures	Rerun from clean build
Selenium test class	Present; archived report has 7/7 with no recorded failures	Rerun and review JVM crash logs
Provisioning automation	Ansible playbook exists but is minimal	Execute safely and validate endpoint
Reliability metrics	No measurements supplied	Define workload and collect measurements before reporting

11. Limitations and Troubleshooting

- Use a known Java 21 installation and verify Maven resolves the same JDK as the terminal/Jenkins agent.

- Wait for a confirmed readiness signal before Selenium begins; a fixed sleep is less reliable than polling a health endpoint.
- Investigate the archived JVM crash logs rather than deleting them before diagnosing the cause.
- Do not report uptime, throughput, or performance values without a repeatable measurement procedure and raw output.

12. Conclusion

This report provides a project-specific, repeatable workflow while separating source-verified configuration from execution that still needs evidence. Final completion should be claimed only after the commands are run in the intended environment and current logs, screenshots, and test reports are retained.

13. References and Evidence

- Project `pom.xml` and `src/test/java/com/busreservation/BusReservationApplicationTests.java`.
- `selenium-tests/pom.xml` and `selenium-tests/src/test/java/com/busreservation/selenium/ReservationSystemE2ETest.java`.
- Archived Surefire summaries included in supplied ZIP (historical evidence only).
- Maven Surefire documentation: https://maven.apache.org/surefire/maven-surefire-plugin/

TASK 15 REPORT

Final End-to-End Release, Documentation and Viva

DevOps Bus Reservation System

Student Name	DIKSHA PARULEKAR
Roll Number	23102A0068

1. Objective

Consolidate architecture, features, DevOps workflow, evidence requirements and release-readiness criteria into a final review document for demonstration and viva.

2. Introduction

The repository describes a Spring Boot bus seat reservation application with a vanilla HTML/CSS/JavaScript frontend, JPA persistence, Spring Security, H2 defaults and a MySQL driver. The checked-in Jenkinsfile provides CI stages for checkout, build, test, package and JAR archiving. Docker and Ansible artifacts exist, but Docker CD is absent from Jenkinsfile and the Ansible health check has caveats. This report marks those boundaries explicitly.

3. Technical Background and Architecture

The system combines a browser frontend with a Spring Boot service and persistence layer. The DevOps toolchain adds Maven, Jenkins, Docker, Ansible and automated test projects, with some delivery stages still requiring implementation.

4. Project-Specific Implementation

Table 15.1 — Technology stack and purpose

Layer/tool	Repository evidence	Purpose
Backend	Java 21; Spring Boot 3.2.4	Application/API framework
Persistence	Spring Data JPA; H2 default; MySQL driver present	ORM and database options
Security	Spring Security; BCrypt described in README	Authentication/security
Frontend	Vanilla HTML5, CSS3, JavaScript Fetch API	Browser UI
Build	Apache Maven	Compile, test, package
CI	Jenkins declarative Jenkinsfile	Checkout/build/test/package/archive
Container	Docker multi-stage Dockerfile	Image build/run definition
Configuration	Ansible inventory + site.yml	Directory task and health URI attempt
UI tests	Selenium WebDriver + JUnit 5 in separate Maven project	Browser-based journey tests

Table 15.2 — Feature and implementation status

Feature/stage	Evidence observed	Status / next action
Registration/login	Described in README; Spring Security dependency present	Verify full UI/API journey
Bus/seat availability	Described in README	Capture real UI/API evidence
Seat booking / duplicate prevention	Described in README	Run transactional/concurrency tests
Booking history/cancellation	Described in README	Verify status transitions and seat release
Jenkins CI	Checkout/build/test/package/archive stages in Jenkinsfile	Pipeline run evidence needed

Feature/stage	Evidence observed	Status / next action
Docker image	Dockerfile present	Build/run evidence needed; port mismatch
Jenkins Docker CD	Not present in Jenkinsfile	Proposed future work
Ansible configuration	Minimal playbook and local inventory present	Run and improve health gate
Automated tests	Application and Selenium test source + archived reports	Rerun; archived reports are historical

5. Implementation Notes

The Git metadata includes task-specific branches for tasks 01–15, but branch names alone do not prove completion. The matrix below is a repository-evidence summary and should be updated with official rubric wording if the official task list is supplied.

6. Diagram(s)

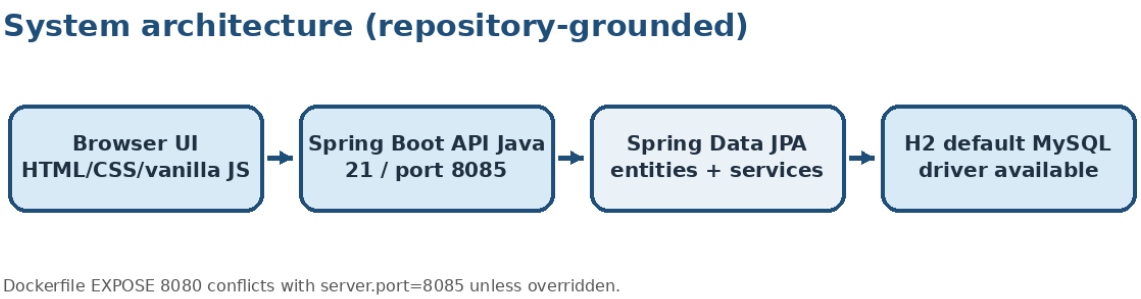


Figure 15.1 — Overall system architecture

This architecture is derived from the README, Maven dependencies, application.properties and Dockerfile. Verify actual frontend resource routes and database deployment settings before release.

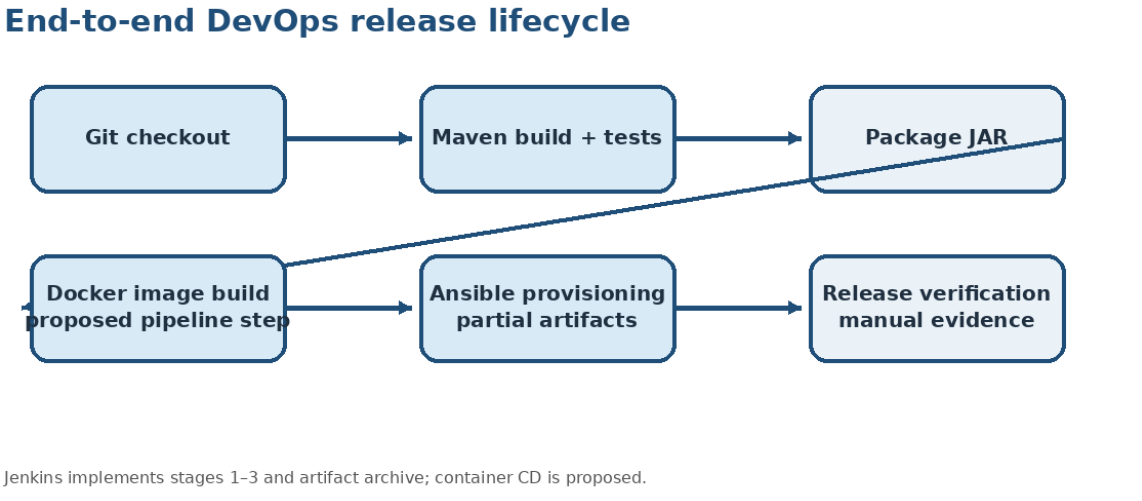


Figure 15.2 — End-to-end DevOps release lifecycle

The first CI stages are implemented in Jenkinsfile. Docker build/push/deploy and robust provisioning/health gates remain proposed unless separately implemented and evidenced.

7. Configuration / Command Tables

Table 15.3 — Summary matrix of all 15 official tasks

Task	Topic	Repository evidence/status
01	Problem scope	Earlier task; source branch/log suggests task exists
02	Agile planning	Earlier task; branch/log suggests task exists

Task	Topic	Repository evidence/status
03	Requirements and architecture	Documentation path exists
04	Git repository setup	Repository metadata exists
05	Feature branching	Feature/task branches appear in Git metadata
06	MVP completion	Application source and tests present
07	Jenkins CI	Jenkinsfile and CI documentation present
08	Pipeline deployment	Pipeline deployment documentation/branch; actual Jenkinsfile has no deploy stage
09	Selenium testing	Selenium test project present; archived reports included
10	Jenkins testing	Jenkinsfile has Maven test stage; Selenium stage not in current Jenkinsfile
11	Docker lifecycle	Dockerfile and task documentation present; port mismatch
12	Jenkins-Docker CD	No Docker build/push/deploy stages in Jenkinsfile
13	Configuration management	Ansible inventory/playbook present; minimal health check
14	Provisioning/reliability	Test source and historical reports present; fresh validation required
15	Final release documentation	This report documents release evidence and gaps

Table 15.4 — Release-readiness checklist

Check	Required evidence/action	Current status
Source/version reviewed	Inspect git status, branch, commit and README	Pending release operator
Build	Fresh Maven clean package succeeds	Pending current run
Tests	Application and Selenium tests pass in current environment	Historical reports only; rerun
Port consistency	Application, Docker and health probe use one agreed port	Action required
Docker image	Build, inspect, run and logs retained	Pending
Jenkins artifact	JAR archived for a successful build	Configured; verify run
Secrets	No credentials committed; Jenkins credentials configured	Review required
Ansible	Syntax check, execution and health result validated	Pending
Documentation	Setup, commands, limitations and evidence updated	This report is draft evidence guide
Rollback	Previous artifact/image and rollback steps identified	Proposed
Viva/demo	Screenshots and live commands prepared	Pending

8. Step-by-Step Procedure

1. Record repository URL, branch, commit hash and working-tree status.
2. Verify JDK/Maven versions and run a clean build plus application tests.
3. Run the Selenium suite against a ready application; save current reports.
4. Choose and document the application port, then align Docker run mapping and health-check URL.
5. Run Jenkins and retain stage logs plus the archived JAR; do not infer a run from the Jenkinsfile alone.
6. Build/run the Docker image and test the deployed application in a non-production environment.
7. Execute Ansible syntax check and playbook; ensure a failed health probe is treated as a release failure if required.
8. Complete the release checklist, capture genuine screenshots, rehearse viva answers, and list limitations honestly.

9. Testing, Validation and Task Status

Table 15.5 — Release validation status

Area	Evidence basis	Required action

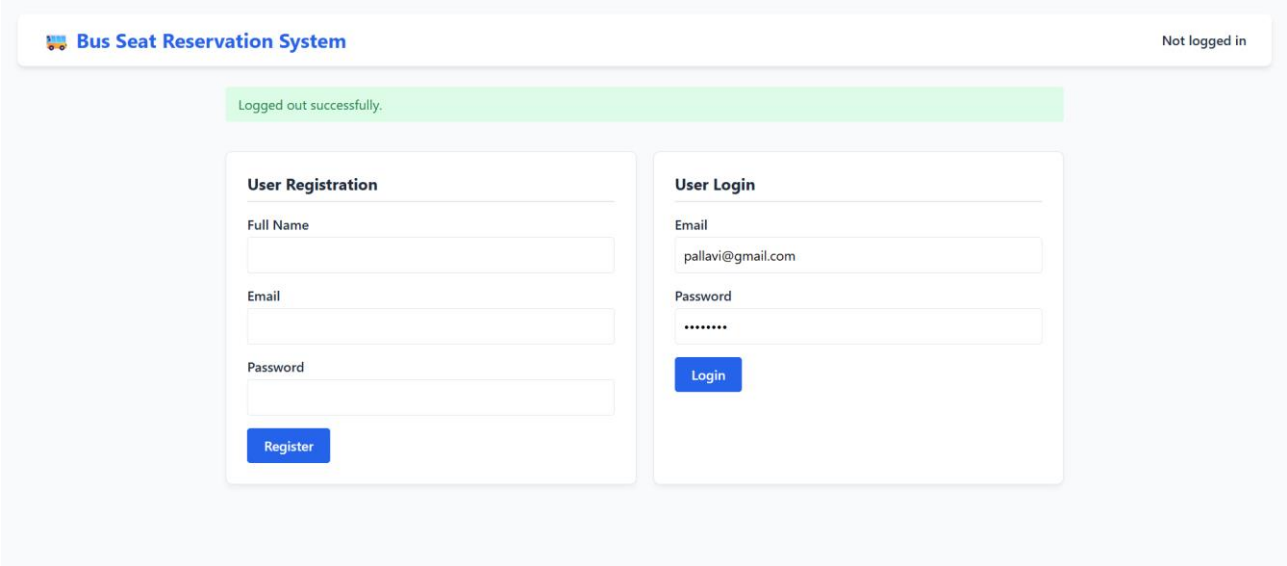
Area	Evidence basis	Required action
Architecture	Grounded in source/config/docs	Review before final presentation
CI	Jenkins stages are defined	Fresh Jenkins run not supplied
Test evidence	Archive contains 5 app tests + 7 Selenium tests, zero recorded failures/errors	Historical; rerun
Release readiness	Port mismatch and health-check gap identified	Not release-ready until resolved/evidenced
Performance/security audit	No benchmark or formal security audit supplied	Not measured / not verified

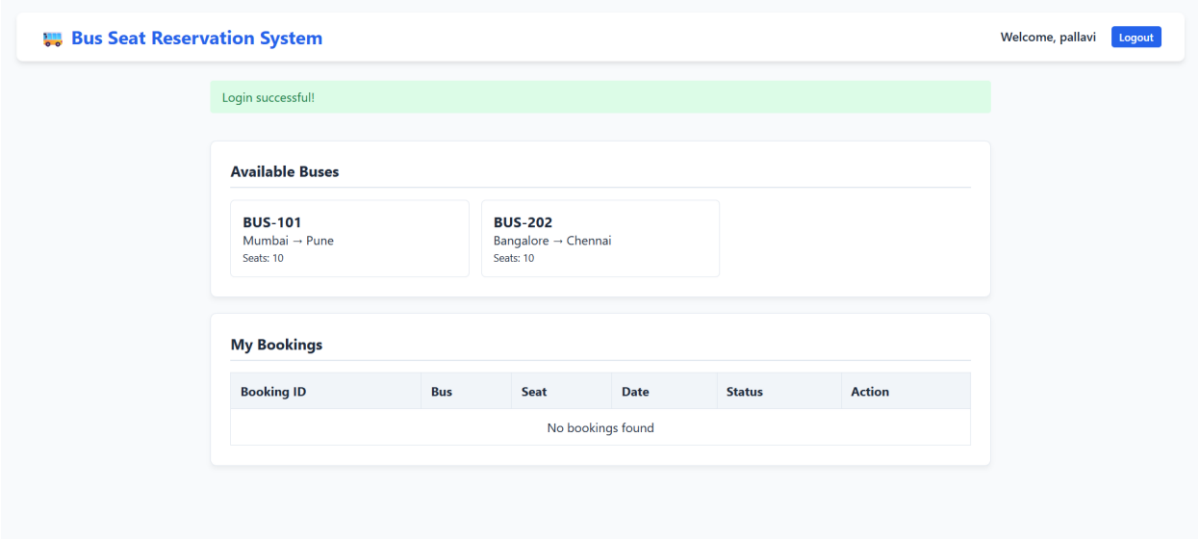
10. Viva Questions and Concise Answers

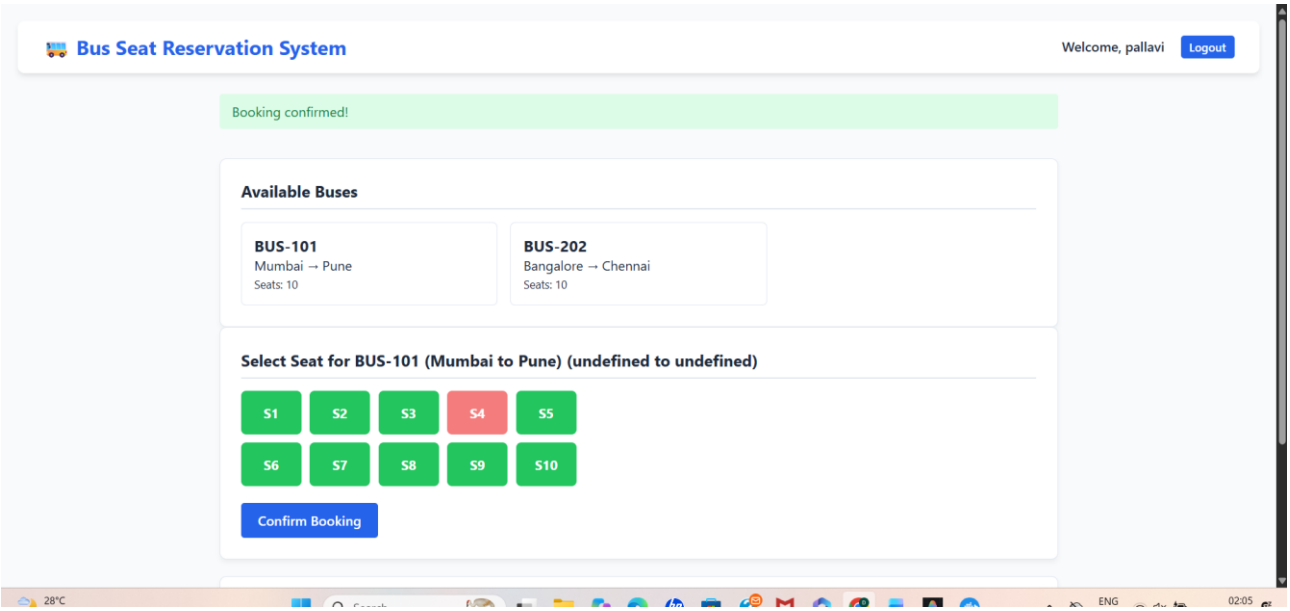
Table 15.6 — Viva preparation

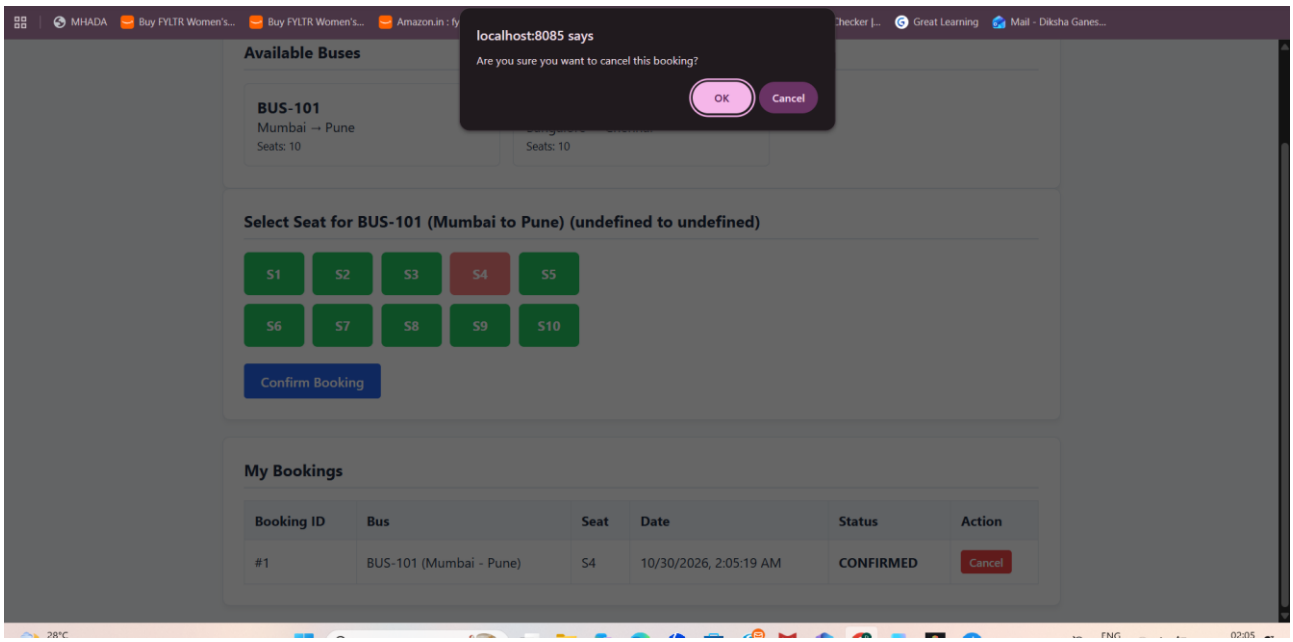
Question	Answer
What is DevOps?	A practice combining development and operations through collaboration, automation and feedback.
What is CI?	Continuous Integration frequently builds and tests changes after source integration.
What does the Jenkinsfile do here?	Checks out source, compiles, tests, packages, and archives JAR artifacts.
Is Docker deployment automated in Jenkins?	Not in the supplied Jenkinsfile; Docker CD is proposed, not implemented there.
Why use a multi-stage Docker build?	It separates Maven/JDK build tooling from the smaller runtime image.
What is the port mismatch?	Dockerfile exposes 8080 while application.properties sets server.port=8085; align or override explicitly.
What does Ansible do currently?	Creates /opt/bus_reservation, attempts a health URI check, and prints a debug message.
Why is ignore_errors important?	A failed health check may not fail the play, so the debug message cannot prove health.
Why H2?	It supports lightweight development/testing with an in-memory default datasource.
What evidence proves tests passed?	A fresh command exit code and current Surefire reports; archived reports are historical.
What is a rollback?	A controlled return to a known previous artifact/image/configuration after failed release validation.
What is the next priority?	Resolve port/health-check consistency, rerun tests, and only then add gated Docker deployment.

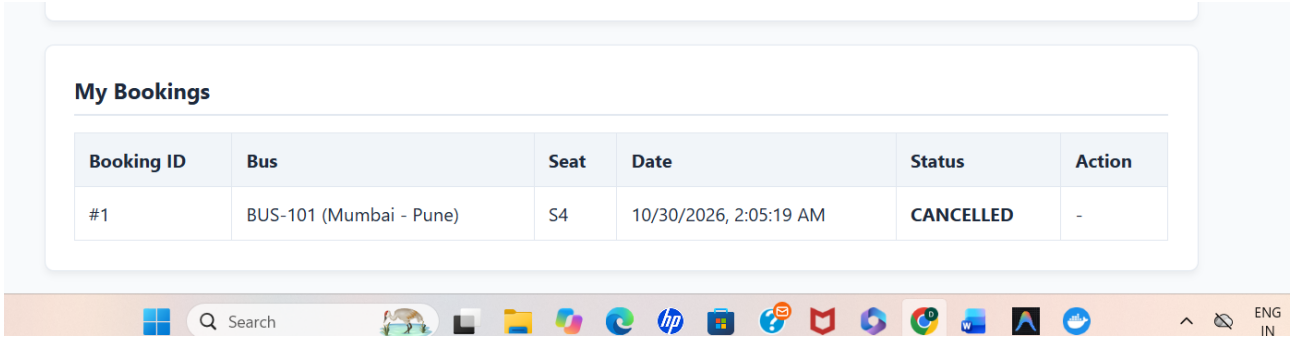
10. Screenshot Placeholders and Evidence Instructions











11. Results and Implementation Status

Table 15.7 — Implementation status summary

Area	Observed status	Evidence/action needed
Core application stack	README/POM/config inspected	Demonstrate actual workflows
CI	Checkout/build/test/package/archive defined	Run and retain Jenkins evidence
Docker	Dockerfile present; app port mismatch identified	Resolve mismatch and validate image
Ansible	Minimal inventory/playbook present	Improve health gate and verify run
Release metrics	No verified uptime/performance figures supplied	Do not state unmeasured metrics

12. Limitations and Troubleshooting

- Treat badges in README as documentation text, not independent proof of current build status.
- A release should not proceed while app port, Docker EXPOSE/mapping and Ansible health URL disagree.
- Add Docker/registry/deployment stages only with credential management, immutable tags, health gates and rollback.
- Use real screenshots and logs; do not replace evidence with diagrams or placeholder text.

13. Conclusion

This report provides a project-specific, repeatable workflow while separating source-verified configuration from execution that still needs evidence. Final completion should be claimed only after the commands are run in the intended environment and current logs, screenshots, and test reports are retained.

14. References and Evidence

- Project `README.md`, `pom.xml`, `Jenkinsfile`, `Dockerfile`.
- `src/main/resources/application.properties`.
- `ansible/inventory.ini`, `ansible/site.yml`, `selenium-tests/` and archived Surefire reports.
- Spring Boot documentation: https://docs.spring.io/spring-boot/docs/3.2.4/reference/html/
- Jenkins Pipeline: https://www.jenkins.io/doc/book/pipeline/
- Docker docs: https://docs.docker.com/
- Ansible docs: https://docs.ansible.com/ansible/latest/